

Team 2: LSTM and CNN Composer Classification

511: Neural Networks and Deep Learning Final Project

✓ Setup and Library Installation

The dataset for this project originates from the Kaggle "MIDI Classic Music" corpus. To facilitate seamless collaboration across different cloud environments (Google Colab and Databricks) without requiring local API keys, the dataset has been hosted on a shared Google Drive. This cell uses the gdown library to securely download and extract the raw MIDI files into the local cluster's temporary storage.

```
# Required libraries
%pip install pretty_midi music21 seaborn kaggle pandas matplotlib tqdm

Collecting pretty_midi
  Downloading pretty_midi-0.2.11.post0-py3-none-any.whl.metadata (4.0
Requirement already satisfied: music21 in /usr/local/lib/python3.12/d
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/d
Requirement already satisfied: kaggle in /usr/local/lib/python3.12/di
Requirement already satisfied: pandas in /usr/local/lib/python3.12/di
Requirement already satisfied: matplotlib in /usr/local/lib/python3.1
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist
Requirement already satisfied: numpy>=1.7.0 in /usr/local/lib/python3
Collecting mido>=1.1.16 (from pretty_midi)
  Downloading mido-1.3.3-py3-none-any.whl.metadata (6.4 kB)
Requirement already satisfied: six in /usr/local/lib/python3.12/dist-
Requirement already satisfied: importlib_resources in /usr/local/lib/
Requirement already satisfied: chardet in /usr/local/lib/python3.12/d
Requirement already satisfied: joblib in /usr/local/lib/python3.12/di
Requirement already satisfied: jsonpickle in /usr/local/lib/python3.1
Requirement already satisfied: more-itertools in /usr/local/lib/pytho
Requirement already satisfied: requests in /usr/local/lib/python3.12/
Requirement already satisfied: webcolors>=1.5 in /usr/local/lib/pytho
Requirement already satisfied: bleach in /usr/local/lib/python3.12/di
Requirement already satisfied: jupyter in /usr/local/lib/python3.12/
Requirement already satisfied: kagglesdk<1.0,>=0.1.20 in /usr/local/l
Requirement already satisfied: packaging in /usr/local/lib/python3.12
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/
Requirement already satisfied: python-dateutil in /usr/local/lib/pyth
Requirement already satisfied: python-slugify in /usr/local/lib/pytho
Requirement already satisfied: urllib3>=1.15.1 in /usr/local/lib/pyth
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/pytho
```

```

Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/pyt
Requirement already satisfied: cycycler>=0.10 in /usr/local/lib/python3
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/py
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/py
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/pyt
Requirement already satisfied: webencodings in /usr/local/lib/python3
Requirement already satisfied: markdown-it-py>=1.0 in /usr/local/lib/
Requirement already satisfied: mdit-py-plugins in /usr/local/lib/pyth
Requirement already satisfied: nbformat in /usr/local/lib/python3.12/
Requirement already satisfied: pyyaml in /usr/local/lib/python3.12/di
Requirement already satisfied: text-unidecode>=1.3 in /usr/local/lib/
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/p
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.1
Requirement already satisfied: fastjsonschema>=2.15 in /usr/local/lib
Requirement already satisfied: jsonschema>=2.6 in /usr/local/lib/pyth
Requirement already satisfied: jupyter-core!=5.0.*,>=4.12 in /usr/loc
Requirement already satisfied: traitlets>=5.1 in /usr/local/lib/pytho
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python
Requirement already satisfied: jsonschema-specifications>=2023.03.6 i
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/
Requirement already satisfied: rpds-py>=0.25.0 in /usr/local/lib/pyth
Requirement already satisfied: platformdirs>=2.5 in /usr/local/lib/py
Requirement already satisfied: typing-extensions>=4.4.0 in /usr/local
Downloading pretty_midi-0.2.11.post0-py3-none-any.whl (5.6 MB)
5.6/5.6 MB 18.8 MB/s eta
Downloading mido-1.3.3-py3-none-any.whl (54 kB)
5.1 6/5.1 6.1 KB/s 1 MB/s eta

```

```

# Imports
%pip install gdown
import gdown
import os
import zipfile

# Define paths
data_dir = "./data/midi_data"
os.makedirs(data_dir, exist_ok=True)
zip_path = os.path.join(data_dir, "midi-classic-music.zip")

# Download from Google Drive
file_id = '1QJPbVi6q2QPIiD47ZBf9tDxhM40N-7mj'
# Corrected URL for direct download
url = f'https://drive.google.com/uc?id={file_id}'

print("Downloading dataset from Google Drive...")
gdown.download(url, zip_path, quiet=False)

# Unzip the file

```

```

print(f"\nExtracting {zip_path}...")
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(data_dir)

print(f"Extraction complete! Files are located in: {data_dir}")

import os
print(os.listdir("./data/midi_data"))

import os

target_composers = ["Bach", "Beethoven", "Chopin", "Mozart"]
data_dir = "./data/midi_data"

for composer in target_composers:
    composer_path = os.path.join(data_dir, composer)
    if os.path.exists(composer_path):
        file_count = len([f for f in os.listdir(composer_path) if f.endswith('.mid')])
        print(f"{composer}: {file_count} MIDI files")
    else:
        print(f"{composer}: folder not found – check exact name")

```

```

Requirement already satisfied: gdown in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: requests[socks] in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: PySocks!=1.5.7,>=1.5.6 in /usr/local/lib/python3.12/dist-packages
Downloading dataset from Google Drive...

```

```

Downloading...
From (original): https://drive.google.com/uc?id=1QJPbVi6q2QPIiD47ZBf9tI
From (redirected): https://drive.google.com/uc?id=1QJPbVi6q2QPIiD47ZBf9tI
To: /content/data/midi_data/midi-classic-music.zip
100%|██████████| 36.4M/36.4M [00:00<00:00, 51.9MB/s]

```

```

Extracting ./data/midi_data/midi-classic-music.zip...
Extraction complete! Files are located in: ./data/midi_data
['Resch', 'Kuhlau Sonatina op55 n3 1mov.mid', 'Vivaldi', 'Schoenberg',
Bach: 122 MIDI files
Beethoven: 133 MIDI files
Chopin: 136 MIDI files
Mozart: 90 MIDI files

```

✓ Data Filtering

The original corpus contains compositions from 175 different composers. Per the project requirements, our predictive models will focus on classifying four specific classical masters: **Bach, Beethoven, Chopin, and Mozart**.

This script iterates through the extracted directory, retains only the target composers, and removes the remaining directories to optimize storage and memory usage.

```
# Filter for specific composers
import os
import shutil

target_composers = ['bach', 'beethoven', 'chopin', 'mozart']

# Find where the composer folders are
base_midi_path = data_dir
if "midi_classic_music" in os.listdir(data_dir):
    base_midi_path = os.path.join(data_dir, "midi_classic_music")

all_folders = [f for f in os.listdir(base_midi_path) if os.path.isdir(f)]
print(f"Found {len(all_folders)} total composer folders.")

# Delete non-target folders
removed_count = 0
for folder in all_folders:
    # Use lowercase for safer string matching
    if folder.lower() not in target_composers:
        shutil.rmtree(os.path.join(base_midi_path, folder))
        removed_count += 1

print(f"Removed {removed_count} folders. Kept target composers only.")
```

```
Found 138 total composer folders.
Removed 134 folders. Kept target composers only.
```

✓ Data Cleaning

Raw data scraped from the internet frequently contains corrupted, truncated, or empty files. Feeding invalid files into deep learning pipelines causes fatal runtime errors during feature extraction.

This cell acts as a quality control gateway: it attempts to parse every `.mid` and `.midi` file using `pretty_midi`, automatically deleting files that are structurally unreadable or contain zero musical notes.

Before exploring or training on the data, we must address class imbalance. If one composer has 1,000 files and another has only 200, our CNN and LSTM models may simply learn to always predict the majority class.

This step calculates the size of the smallest composer class and randomly undersamples the majority classes to match it. A fixed random seed (42) is utilized to guarantee that all team members generate the exact same balanced dataset.

```
# Data Cleaning Pipeline
import pretty_midi
import glob
from tqdm import tqdm
import random
import os
from collections import Counter

# Find all .mid and .midi files in the remaining folders
midi_files = glob.glob(f"{base_midi_path}/**/*.mid", recursive=True) +
              glob.glob(f"{base_midi_path}/**/*.midi", recursive=True)

print(f"Found {len(midi_files)} potential MIDI files for our 4 composers")

valid_files = []
corrupt_files = []

# Loop and validate
for file_path in tqdm(midi_files, desc="Validating MIDI files"):
    try:
        # Parse the MIDI file
        midi_data = pretty_midi.PrettyMIDI(file_path)

        # Check if the file contains instruments/notes
        if len(midi_data.instruments) == 0:
```

```

        raise ValueError("No instruments found.")

    total_notes = sum(len(instrument.notes) for instrument in mid:
    if total_notes == 0:
        raise ValueError("No notes found.")

    # Add to valid list if it passes
    valid_files.append(file_path)

except Exception as e:
    corrupt_files.append((file_path, str(e)))
    # Delete the bad file
    os.remove(file_path)

print(f"\nCleaning Summary")
print(f"Valid Files: {len(valid_files)}")
print(f"Corrupt/Empty Files Removed: {len(corrupt_files)}")
if corrupt_files:
    print(f"Example error: {corrupt_files[0]}")

# Handling Class Imbalance via Undersampling
# Group valid files by composer
composer_files = {composer: [] for composer in target_composers}
composer_counts = Counter()

for file_path in valid_files:
    # Extract composer name from the folder path and lowercase it
    composer = os.path.basename(os.path.dirname(file_path)).lower()
    if composer in target_composers:
        composer_files[composer].append(file_path)
        composer_counts[composer] += 1

print("Original Clean File Counts")
for comp, count in composer_counts.items():
    print(f"{comp.capitalize()}: {count}")

# Minimum class size
min_class_size = min(composer_counts.values())
print(f"\nBalancing dataset to {min_class_size} files per composer")

# Randomly undersample the majority classes
balanced_valid_files = []

# Random seed so everyone gets the exact same files every time it runs
random.seed(42)

for composer, files in composer_files.items():
    # Randomly select 'min_class_size' files from this composer's list

```

```

        sampled_files = random.sample(files, min_class_size)
        balanced_valid_files.extend(sampled_files)

print(f"\nBalanced Dataset Summary")
print(f"Total Balanced Files: {len(balanced_valid_files)}")

# Overwrite the original valid_files list
valid_files = balanced_valid_files

Found 1630 potential MIDI files for our 4 composers.
Validating MIDI files:  0%|          | 0/1630 [00:00<?, ?it/s]/usr/lo
    warnings.warn(
Validating MIDI files: 100%|██████████| 1630/1630 [03:30<00:00,  7.76i
Cleaning Summary
Valid Files: 1626
Corrupt/Empty Files Removed: 4
Example error: ('./data/midi_data/Buxethude Buxwv161 Passcaglia.mid',
Original Clean File Counts
Beethoven: 132
Bach: 122
Mozart: 90
Chopin: 136

Balancing dataset to 90 files per composer

Balanced Dataset Summary
Total Balanced Files: 360

```

✓ Exploratory Data Analysis (EDA)

Before transforming the MIDI files into high-dimensional sequences (for the LSTM) or 2D piano-roll matrices (for the CNN), we extract high-level statistical features to understand the underlying distributions.

This script loops through the balanced dataset and extracts metrics such as composition length, note density (notes per second), estimated tempo (BPM), and average pitch.

```

# EDA metrics
import pandas as pd
import numpy as np

eda_data = []

for file_path in tqdm(valid_files, desc="Extracting EDA Features"):

```

```

try:
    composer = os.path.basename(os.path.dirname(file_path)).capitalize()
    midi_data = pretty_midi.PrettyMIDI(file_path)

    # Total length of piece in seconds
    length_sec = midi_data.get_end_time()

    # Total number of notes across all tracks
    total_notes = sum(len(instrument.notes) for instrument in midi_data.instruments)

    # Estimated Tempo (BPM)
    tempo = midi_data.estimate_tempo()

    # Average pitch and pitch range
    all_pitches = []
    for instrument in midi_data.instruments:
        # Ignoring drum tracks for this analysis
        if not instrument.is_drum:
            all_pitches.extend([note.pitch for note in instrument.notes])

    if all_pitches:
        avg_pitch = np.mean(all_pitches)
        pitch_range = np.max(all_pitches) - np.min(all_pitches)
    else:
        avg_pitch = 0
        pitch_range = 0

    # Append to list
    eda_data.append({
        "Composer": composer,
        "Length_Seconds": length_sec,
        "Total_Notes": total_notes,
        "Notes_Per_Second": total_notes / length_sec if length_sec > 0 else 0,
        "Estimated_BPM": tempo,
        "Average_Pitch": avg_pitch,
        "Pitch_Range": pitch_range
    })
except Exception:
    continue # Skip files that act up during deep extraction

df_eda = pd.DataFrame(eda_data)
display(df_eda.head())

```

```

Extracting EDA Features: 2%|| | 9/360 [00:04<02:14, 2.61it/s]
warnings.warn(
Extracting EDA Features: 100%|██████████| 360/360 [01:06<00:00, 5.39it/s]

  Composer  Length_Seconds  Total_Notes  Notes_Per_Second  Estimated_BPM
0      Bach             618.158060           4421             7.151893           160.82790

```


	Composer	Notes_Per_Second	Piece_Length_Seconds	Tempo_BPM	Estimated_Tempo
1	Bach	745.165200	4493	6.029535	246.25040
2	Bach	515.662760	3367	6.529461	218.6893
3	Bach	857.375929	20062	23.399304	260.79834
4	Bach	682.367570	5930	8.690331	164.09395

✓ Visualization

Using the extracted features, this cell generates a series of visualizations to explore structural differences between the four composers.

By analyzing note density, piece length, and tempo distributions, we can visually identify if certain composers have distinct stylistic signatures that our neural networks will likely rely on for classification.

```
# Plotting the Data
import matplotlib.pyplot as plt
import seaborn as sns

# Visual style
sns.set_theme(style="whitegrid")
plt.figure(figsize=(16, 10))

# Distribution of Classes (Imbalance check)
plt.subplot(2, 2, 1)
sns.countplot(data=df_eda, x="Composer", palette="viridis")
plt.title("Number of Pieces per Composer")

# Notes per Second (Complexity/Density)
plt.subplot(2, 2, 2)
sns.boxplot(data=df_eda, x="Composer", y="Notes_Per_Second", palette='
plt.title("Note Density (Notes per Second) by Composer")
plt.ylim(0, df_eda["Notes_Per_Second"].quantile(0.95))

# Piece Lengths
plt.subplot(2, 2, 3)
sns.violinplot(data=df_eda, x="Composer", y="Length_Seconds", palette=
plt.title("Distribution of Piece Lengths (Seconds)")
plt.ylim(0, 1000)

# Estimated Tempo
```

```
plt.subplot(2, 2, 4)
sns.kdeplot(data=df_eda, x="Estimated_BPM", hue="Composer", fill=True,
plt.title("Tempo (BPM) Density by Composer")

plt.tight_layout()
plt.show()
```

/tmp/ipykernel_2557/1593933281.py:11: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in a future version. Use `hue=palette\_name` instead.

```
sns.countplot(data=df_eda, x="Composer", palette="viridis")
```

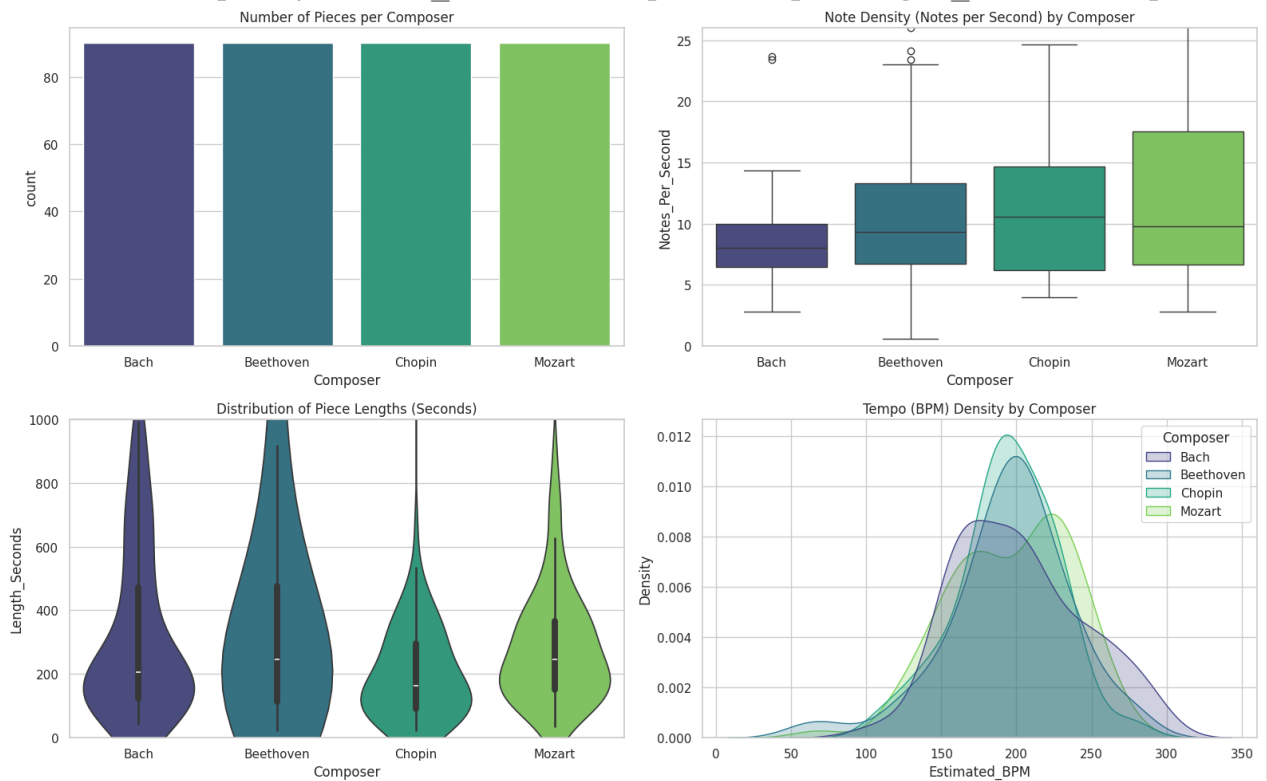
/tmp/ipykernel_2557/1593933281.py:16: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in a future version. Use `hue=palette\_name` instead.

```
sns.boxplot(data=df_eda, x="Composer", y="Notes_Per_Second", palette=
/tmp/ipykernel_2557/1593933281.py:22: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in a future version. Use `hue=palette\_name` instead.

```
sns.violinplot(data=df_eda, x="Composer", y="Length_Seconds", palette=
```



✓ **Observation on the Data:**

@Christian To include comment here

Next Steps: We will evaluate the classification task using two models:

1 - LSTM

2 - CNN

✓ **LONG SHORT-TERM MEMORY (LSTM)**

Recurrent Neural Network (RNN) Model

```
# LSTM Feature Extraction with Time Stretching Augmentation
# Converts each MIDI file into a sequence of note/chord "tokens" using
# Each token represents either a single note (pitch) or a chord (multiple
# played together), which the LSTM will learn sequential patterns from
#
```

```
# AUGMENTATION: For each file, we also create time-stretched versions
# to help the model handle Mozart's wide variability in notes-per-second
```

```
from music21 import converter, note, chord
from tqdm import tqdm
import glob
import copy
import os # Added import os
```

```
# Reconstruct valid_files if it doesn't exist (e.g., after kernel res
if 'valid_files' not in dir():
    print("Reconstructing valid_files from filesystem...")
    data_dir = "./data/midi_data" # Re-define data_dir
    os.makedirs(data_dir, exist_ok=True) # Ensure data_dir exists
    base_midi_path = data_dir
    if "midi_classic_music" in os.listdir(data_dir):
        base_midi_path = os.path.join(data_dir, "midi_classic_music")

    valid_files = glob.glob(f"{base_midi_path}/**/*.mid", recursive=True) +
        glob.glob(f"{base_midi_path}/**/*.midi", recursive=True)
    print(f"Found {len(valid_files)} MIDI files from filesystem")
```

```
def time_stretch_midi(midi_obj, factor):
    """Apply time stretching by scaling all note durations."""
    stretched = copy.deepcopy(midi_obj)
    for element in stretched.flat.notes:
        element.quarterLength *= factor
    return stretched
```

```
def extract_note_sequence(midi_obj):
    """Parse a MIDI object and return a list of note/chord tokens."""
    notes_to_parse = None

    parts = midi_obj.getElementsByClass('Part')
    if parts:
        notes_to_parse = parts[0].recurse()
    else:
        notes_to_parse = midi_obj.flat.notes

    sequence = []
    for element in notes_to_parse:
        if isinstance(element, note.Note):
            sequence.append(str(element.pitch))
        elif isinstance(element, chord.Chord):
            sequence.append('.'.join(str(n) for n in element.normalOrder))

    return sequence
```

```

# Run extraction with time stretching augmentation
lstm_data = [] # list of (composer, sequence) tuples
failed_files = []
time_stretch_factors = [0.9, 1.0, 1.1] # 0.9x (faster), original, 1.1x (slower)

for file_path in tqdm(valid_files, desc="Extracting sequences with time stretching"):
    try:
        composer = os.path.basename(os.path.dirname(file_path)).capitalize()

        # Parse the original MIDI file once
        midi_original = converter.parse(file_path)

        # Extract sequences for each time-stretched version
        for factor in time_stretch_factors:
            if factor == 1.0:
                # Use original without copying
                midi_obj = midi_original
            else:
                # Apply time stretching
                midi_obj = time_stretch_midi(midi_original, factor)

            sequence = extract_note_sequence(midi_obj)
            if len(sequence) > 0:
                lstm_data.append((composer, sequence))

    except Exception as e:
        failed_files.append((file_path, str(e)))
        continue

print(f"\nSuccessfully extracted sequences from {len(lstm_data)} augmented files (Original files: {len(valid_files)}), Augmentation factor: {len(time_stretch_factors)}")
print(f"Failed to extract: {len(failed_files)} files")
if failed_files:
    print(f"Example failure: {failed_files[0]}")

```

```

Extracting sequences with time stretching:  0%|          | 0/360 [00:00]
return self.iter().getElementsByClass(classFilterList)
Extracting sequences with time stretching: 11%|█         | 41/360 [00:01]
warnings.warn(
Extracting sequences with time stretching: 26%|██        | 95/360 [00:02]
warnings.warn(
Extracting sequences with time stretching: 29%|███       | 105/360 [00:02]
warnings.warn(
/usr/local/lib/python3.12/dist-packages/music21/midi/translate.py:922:
warnings.warn(
Extracting sequences with time stretching: 34%|████      | 122/360 [00:03]
warnings.warn(
Extracting sequences with time stretching: 34%|████      | 124/360 [00:03]
warnings.warn(

```

```

Extracting sequences with time stretching: 35%|██████      | 126/360 [
  warnings.warn(
Extracting sequences with time stretching: 40%|██████      | 145/360 [
  warnings.warn(
Extracting sequences with time stretching: 42%|██████      | 150/360 [
  warnings.warn(
Extracting sequences with time stretching: 42%|██████      | 152/360 [
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/music21/midi/translate.py:922
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/music21/midi/translate.py:922
  warnings.warn(
Extracting sequences with time stretching: 50%|██████      | 179/360 [
  warnings.warn(
Extracting sequences with time stretching: 51%|██████      | 184/360 [
  warnings.warn(
Extracting sequences with time stretching: 51%|██████      | 185/360 [
  warnings.warn(
Extracting sequences with time stretching: 53%|██████      | 190/360 [
  warnings.warn(
Extracting sequences with time stretching: 54%|██████      | 193/360 [
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/music21/midi/translate.py:922
  warnings.warn(
Extracting sequences with time stretching: 55%|██████      | 197/360 [
  warnings.warn(
Extracting sequences with time stretching: 56%|██████      | 200/360 [
  warnings.warn(
Extracting sequences with time stretching: 62%|██████      | 224/360 [
  warnings.warn(
Extracting sequences with time stretching: 63%|██████      | 228/360 [
  warnings.warn(
Extracting sequences with time stretching: 68%|██████      | 245/360 [
  warnings.warn(
Extracting sequences with time stretching: 72%|██████      | 258/360 [
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/music21/midi/translate.py:922
  warnings.warn(
Extracting sequences with time stretching: 74%|██████      | 267/360 [
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/music21/midi/translate.py:922
  warnings.warn(
Extracting sequences with time stretching: 77%|██████      | 276/360 [
  warnings.warn(

```

```

# =====
# LSTM SEQUENCE ENCODING
# =====

# LSTM Sequence Encoding
# Builds a vocabulary of all unique note/chord tokens seen across the
# then converts each sequence into a list of integers the LSTM can coi

```

```

from collections import Counter

# Build vocabulary from all sequences
all_tokens = [token for (composer, sequence) in lstm_data for token in sequence]
token_counts = Counter(all_tokens)
vocab = sorted(token_counts.keys())

token_to_int = {token: i for i, token in enumerate(vocab)}
int_to_token = {i: token for token, i in token_to_int.items()}

vocab_size = len(vocab)
print(f"Vocabulary size: {vocab_size} unique note/chord tokens")

# Encode each sequence as a list of integers
encoded_data = [(composer, [token_to_int[t] for t in sequence]) for composer, sequence in lstm_data]

# Quick sanity check
if len(encoded_data) > 0:
    print(f"\nExample encoded sequence (first 20 tokens) from a {encoded_data[0][0]} piece:")
    print(encoded_data[0][1][:20])
else:
    print("\nWARNING: No sequences were successfully encoded. Check the data source.")

# Check sequence length distribution (important for choosing a fixed window size)
seq_lengths = [len(seq) for (_, seq) in encoded_data]
if len(seq_lengths) > 0:
    print(f"\nSequence length stats:")
    print(f"Min: {min(seq_lengths)}, Max: {max(seq_lengths)}, Mean: {sum(seq_lengths)/len(seq_lengths)}")
else:
    print("\nNo sequences available for length analysis.")

```

Vocabulary size: 902 unique note/chord tokens

Example encoded sequence (first 20 tokens) from a Bach piece:
[846, 866, 873, 839, 892, 839, 873, 880, 893, 873, 839, 873, 893, 827,

Sequence length stats:
Min: 29, Max: 5210, Mean: 842

```

# =====
# LSTM TRAINING DATA PREPARATION
# =====

# LSTM Training Data Preparation with Enhanced Random Cropping
# Since sequence lengths vary hugely (29 to 5210 tokens), we slice each
# into fixed-length windows. Each window becomes one training example.

```

```

# with its composer.
#
# AUGMENTATION: Instead of systematic sliding windows, we use RANDOM
# to capture Mozart's sparse AND dense sections equally (not just sta

import numpy as np
import random

SEQUENCE_LENGTH = 100 # number of tokens per training example
WINDOWS_PER_SEQUENCE = 10 # number of random windows to extract per :

random.seed(42) # For reproducibility

X = []
y = []

composers_list = sorted(set(composer for composer, _ in encoded_data))
composer_to_int = {c: i for i, c in enumerate(composers_list)}

for composer, seq in encoded_data:
    label = composer_to_int[composer]

    # Skip sequences that are too short
    if len(seq) < SEQUENCE_LENGTH:
        continue

    # For sequences that are exactly SEQUENCE_LENGTH, take it once
    if len(seq) == SEQUENCE_LENGTH:
        X.append(seq)
        y.append(label)
    else:
        # Extract multiple random windows from longer sequences
        for _ in range(WINDOWS_PER_SEQUENCE):
            # Random starting position
            max_start = len(seq) - SEQUENCE_LENGTH
            start = random.randint(0, max_start)
            window = seq[start:start + SEQUENCE_LENGTH]
            X.append(window)
            y.append(label)

X = np.array(X)
y = np.array(y)

print(f"Composer label mapping: {composer_to_int}")
print(f"Total training windows generated: {len(X)}")
print(f"X shape: {X.shape}, y shape: {y.shape}")

# Check class balance after windowing

```



```

unique, counts = np.unique(y, return_counts=True)
for label_idx, count in zip(unique, counts):
    composer_name = composers_list[label_idx]
    print(f" {composer_name}: {count} windows")

```

```

Composer label mapping: {'Bach': 0, 'Beethoven': 1, 'Chopin': 2, 'Mozart': 3}
Total training windows generated: 10140
X shape: (10140, 100), y shape: (10140,)
Bach: 2670 windows
Beethoven: 2400 windows
Chopin: 2580 windows
Mozart: 2490 windows

```

```

# =====
# LSTM MODEL
# =====

%pip install tensorflow

# Train/Test Split + LSTM Model
from sklearn.model_selection import train_test_split
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
from tensorflow.keras.utils import to_categorical

# Split into train/test (stratified to preserve composer balance)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"Train shape: {X_train.shape}, Test shape: {X_test.shape}")

# One-hot encode labels for multi-class classification
num_classes = len(composers_list)
y_train_cat = to_categorical(y_train, num_classes=num_classes)
y_test_cat = to_categorical(y_test, num_classes=num_classes)

# Build the LSTM model
model = Sequential([
    Embedding(input_dim=vocab_size, output_dim=64, input_length=SEQUENCE_LENGTH),
    LSTM(128, return_sequences=True),
    Dropout(0.3),
    LSTM(64),
    Dropout(0.3),
    Dense(32, activation='relu'),
    Dense(num_classes, activation='softmax')
])

```

```

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

```

```
model.summary()
```

```

Requirement already satisfied: tensorflow in /usr/local/lib/python3.1
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/pytho
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/py
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib
Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in
Requirement already satisfied: google_pasta>=0.1.1 in /usr/local/lib/
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/pyt
Requirement already satisfied: opt_einsum>=2.3.2 in /usr/local/lib/py
Requirement already satisfied: packaging in /usr/local/lib/python3.12
Requirement already satisfied: protobuf>=5.28.0 in /usr/local/lib/pyt
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/
Requirement already satisfied: setuptools in /usr/local/lib/python3.1
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/pyt
Requirement already satisfied: typing_extensions>=3.6.6 in /usr/local
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/
Requirement already satisfied: tensorboard~=2.20.0 in /usr/local/lib/
Requirement already satisfied: keras>=3.10.0 in /usr/local/lib/python
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3
Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in /usr/local/
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/p
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist
Requirement already satisfied: namex in /usr/local/lib/python3.12/dis
Requirement already satisfied: optree in /usr/local/lib/python3.12/di
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/p
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/p
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/pyth
Requirement already satisfied: pillow in /usr/local/lib/python3.12/di
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/pyth
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/py
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/li
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.1
Train shape: (8112, 100), Test shape: (2028, 100)

```

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embeddi
warnings.warn(

```

Model: "sequential"

Layer (type)	Output Shape	Pa
--------------	--------------	----

embedding (Embedding)	?	0 (unb
lstm (LSTM)	?	0 (unb
dropout (Dropout)	?	
lstm_1 (LSTM)	?	0 (unb
dropout_1 (Dropout)	?	
dense (Dense)	?	0 (unb
dense_1 (Dense)	?	0 (unb

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

```
# =====
# LSTM TRAINING
# =====

# Train the LSTM model
history = model.fit(
    X_train, y_train_cat,
    validation_data=(X_test, y_test_cat),
    epochs=30,
    batch_size=64,
    verbose=1
)

# =====
# LSTM TRAINING HISTORY VISUALIZATION
# =====

# Training History Visualization
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Val Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.title('Accuracy Over Epochs')

plt.subplot(1, 2, 2)
```

```
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.title('Loss Over Epochs')

plt.tight_layout()
plt.show()
```

```
Epoch 1/30
127/127  45s 320ms/step - accuracy: 0.5044 - loss
Epoch 2/30
127/127  43s 342ms/step - accuracy: 0.6305 - loss
Epoch 3/30
127/127  40s 313ms/step - accuracy: 0.6557 - loss
Epoch 4/30
127/127  39s 310ms/step - accuracy: 0.6564 - loss
Epoch 5/30
127/127  41s 327ms/step - accuracy: 0.6815 - loss
Epoch 6/30
127/127  42s 332ms/step - accuracy: 0.7083 - loss
Epoch 7/30
127/127  40s 312ms/step - accuracy: 0.5820 - loss
Epoch 8/30
127/127  39s 309ms/step - accuracy: 0.6056 - loss
Epoch 9/30
127/127  40s 316ms/step - accuracy: 0.6584 - loss
Epoch 10/30
127/127  40s 313ms/step - accuracy: 0.7120 - loss
Epoch 11/30
127/127  39s 310ms/step - accuracy: 0.7437 - loss
Epoch 12/30
127/127  40s 314ms/step - accuracy: 0.7531 - loss
Epoch 13/30
127/127  41s 311ms/step - accuracy: 0.7638 - loss
Epoch 14/30
127/127  39s 309ms/step - accuracy: 0.7456 - loss
Epoch 15/30
127/127  41s 312ms/step - accuracy: 0.7138 - loss
Epoch 16/30
127/127  41s 311ms/step - accuracy: 0.7378 - loss
Epoch 17/30
127/127  41s 310ms/step - accuracy: 0.7599 - loss
Epoch 18/30
127/127  40s 312ms/step - accuracy: 0.7653 - loss
Epoch 19/30
127/127  41s 310ms/step - accuracy: 0.7276 - loss
Epoch 20/30
127/127  39s 311ms/step - accuracy: 0.7322 - loss
Epoch 21/30
127/127  39s 309ms/step - accuracy: 0.7511 - loss
Epoch 22/30
```

127/127 ————— 42s 331ms/step - accuracy: 0.7821 - loss
Epoch 23/30

127/127 ————— 39s 311ms/step - accuracy: 0.7643 - loss
Epoch 24/30

127/127 ————— 41s 309ms/step - accuracy: 0.7687 - loss
Epoch 25/30

127/127 ————— 40s 312ms/step - accuracy: 0.7554 - loss
Epoch 26/30

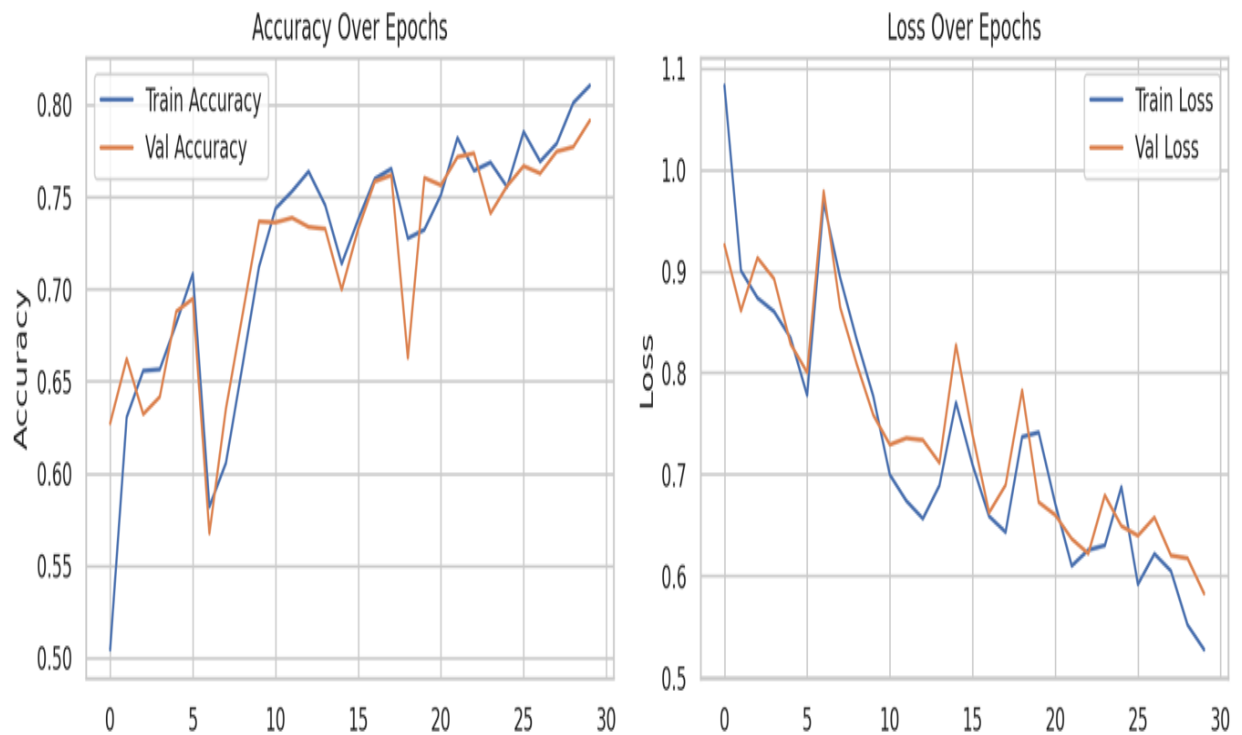
127/127 ————— 42s 329ms/step - accuracy: 0.7854 - loss
Epoch 27/30

127/127 ————— 42s 330ms/step - accuracy: 0.7692 - loss
Epoch 28/30

127/127 ————— 39s 309ms/step - accuracy: 0.7790 - loss
Epoch 29/30

127/127 ————— 41s 309ms/step - accuracy: 0.8010 - loss
Epoch 30/30

127/127 ————— 40s 312ms/step - accuracy: 0.8104 - loss



```
# =====
# LSTM MODEL EVALUATION
# =====

# LSTM Model Evaluation
from sklearn.metrics import classification_report, confusion_matrix,
import numpy as np

# Predict on test set
y_pred_probs = model.predict(X_test)
y_pred = np.argmax(y_pred_probs, axis=1)
```

```

# Overall accuracy
test_accuracy = accuracy_score(y_test, y_pred)
print(f"Test Accuracy: {test_accuracy:.4f}\n")

# Precision, Recall, F1 per composer
print("Classification Report:")
print(classification_report(y_test, y_pred, target_names=composers_list))

# Confusion matrix
print("\nConfusion Matrix:")
cm = confusion_matrix(y_test, y_pred)
print(cm)

# Visualize confusion matrix
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=composers_list, yticklabels=composers_list)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('LSTM Confusion Matrix')
plt.tight_layout()
plt.show()

# Save LSTM results for comparison later
lstm_test_accuracy = test_accuracy
lstm_y_test = y_test
lstm_y_pred = y_pred
lstm_cm = cm

```

64/64  4s 58ms/step

Test Accuracy: 0.7914

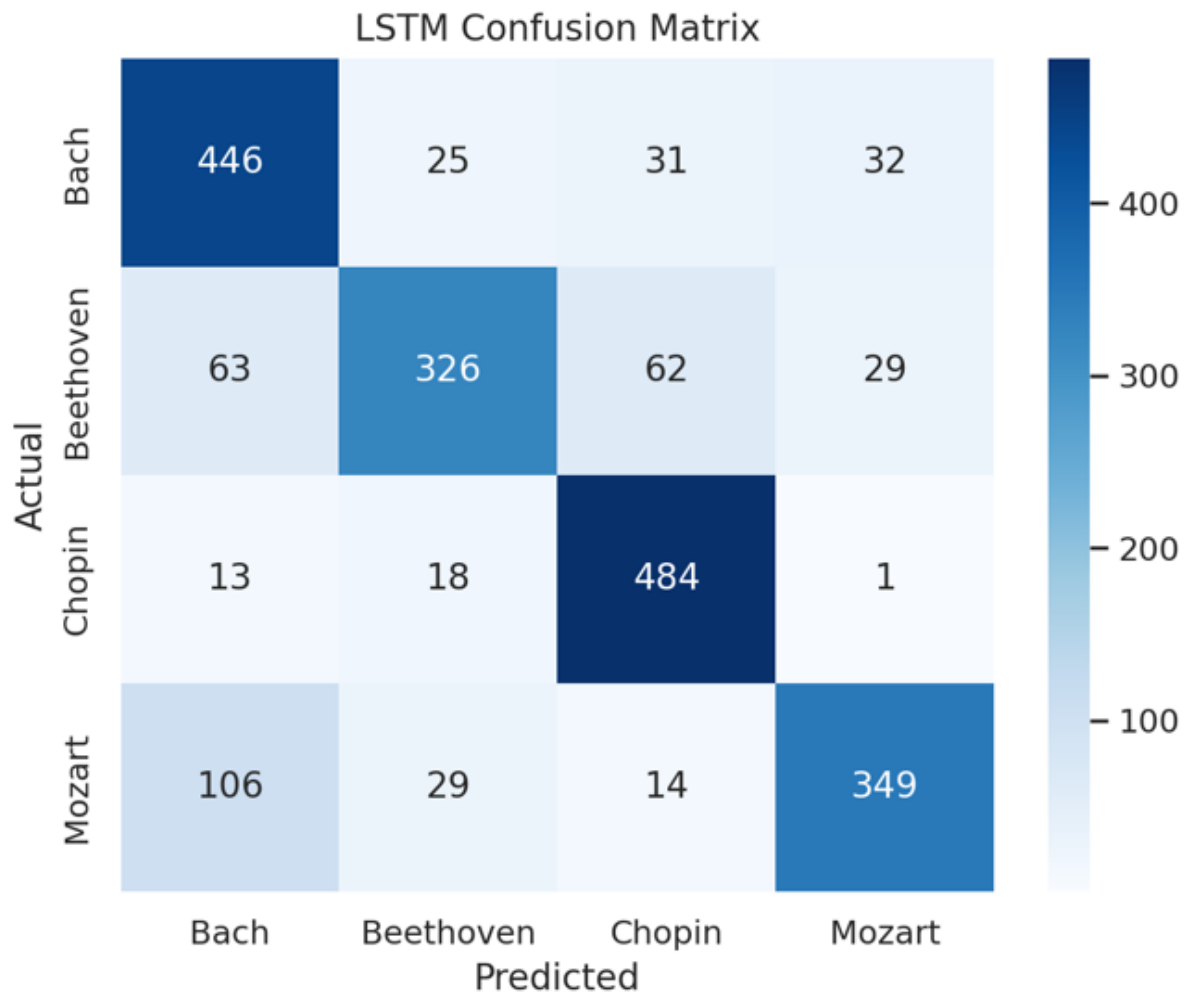
Classification Report:

	precision	recall	f1-score	support
Bach	0.71	0.84	0.77	534
Beethoven	0.82	0.68	0.74	480
Chopin	0.82	0.94	0.87	516
Mozart	0.85	0.70	0.77	498
accuracy			0.79	2028
macro avg	0.80	0.79	0.79	2028
weighted avg	0.80	0.79	0.79	2028

Confusion Matrix:

```
[[446  25  31  32]
```

```
[ 63 326  62  29]
[ 13  18 484   1]
[106  29  14 349]]
```



Observation on the LSTM Model:

@ldrees to comment here

✓ CONVOLUTION NEURAL NETWORK (CNN) MODEL

```
# =====
# CONVOLUTION NEURAL NETWORK (CNN) MODEL
# =====

%pip install pretty_midi seaborn pandas matplotlib tqdm
%pip install tensorflow
```

Requirement already satisfied: pretty_midi in /usr/local/lib/python3.12/dist-packages/pretty_midi-0.2.2-py3.12.egg

Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages/seaborn-0.12.2-py3.12.egg

Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages/pandas-2.0.3-py3.12.egg

Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages/matplotlib-3.7.1-py3.12.egg

Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages/tqdm-4.66.1-py3.12.egg

Requirement already satisfied: numpy>=1.7.0 in /usr/local/lib/python3.12/dist-packages/numpy-1.26.0-py3.12.egg

Requirement already satisfied: mido>=1.1.16 in /usr/local/lib/python3.12/dist-packages/mido-1.2.10-py3.12.egg

Requirement already satisfied: six in /usr/local/lib/python3.12/dist-packages/six-1.16.0-py3.12.egg

Requirement already satisfied: importlib_resources in /usr/local/lib/python3.12/dist-packages/importlib_resources-6.1.1-py3.12.egg

Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages/python-dateutil-2.8.2-py3.12.egg

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages/pytz-2022.7-py3.12.egg

Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages/tzdata-2022.7-py3.12.egg

Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages/contourpy-1.0.7-py3.12.egg

Requirement already satisfied: cycycler>=0.10 in /usr/local/lib/python3.12/dist-packages/cycycler-0.10.0-py3.12.egg

Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages/fonttools-4.22.0-py3.12.egg

Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages/kiwisolver-1.3.1-py3.12.egg

Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages/packaging-20.9-py3.12.egg

Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages/pillow-9.5.0-py3.12.egg

Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages/pyparsing-3.0.9-py3.12.egg

Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-packages/tensorflow-2.12.0-py3.12.egg

Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages/absl-py-1.0.0-py3.12.egg

Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages/astunparse-1.6.3-py3.12.egg

Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages/flatbuffers-24.3.25-py3.12.egg

Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages/gast-0.5.3-py3.12.egg

Requirement already satisfied: google_pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages/google_pasta-0.1.1-py3.12.egg

Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages/libclang-13.0.0-py3.12.egg

Requirement already satisfied: opt_einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages/opt_einsum-3.3.0-py3.12.egg

Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages/packaging-20.9-py3.12.egg

Requirement already satisfied: protobuf>=5.28.0 in /usr/local/lib/python3.12/dist-packages/protobuf-5.28.0-py3.12.egg

Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages/requests-2.31.0-py3.12.egg

Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages/setuptools-68.0.0-py3.12.egg

Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages/six-1.16.0-py3.12.egg

Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dist-packages/termcolor-1.1.0-py3.12.egg

Requirement already satisfied: typing_extensions>=3.6.6 in /usr/local/lib/python3.12/dist-packages/typing_extensions-4.6.2-py3.12.egg

Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-packages/wrapt-1.11.0-py3.12.egg

Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages/grpcio-1.24.3-py3.12.egg

Requirement already satisfied: tensorboard~=2.20.0 in /usr/local/lib/python3.12/dist-packages/tensorboard-2.20.0-py3.12.egg

Requirement already satisfied: keras>=3.10.0 in /usr/local/lib/python3.12/dist-packages/keras-3.10.0-py3.12.egg

Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages/numpy-1.26.0-py3.12.egg

Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-packages/h5py-3.11.0-py3.12.egg

Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3.12/dist-packages/ml_dtypes-0.5.1-py3.12.egg

Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/dist-packages/wheel-0.23.0-py3.12.egg

Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages/rich-13.7.1-py3.12.egg

Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages/namex-0.0.7-py3.12.egg

Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages/optree-0.11.0-py3.12.egg

Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages/charset_normalizer-3.3.0-py3.12.egg

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages/idna-3.6-py3.12.egg

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages/urllib3-2.0.7-py3.12.egg

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages/certifi-2022.9.24-py3.12.egg

Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/dist-packages/markdown-3.4.3-py3.12.egg

Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages/pillow-9.5.0-py3.12.egg

Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages/tensorboard-data-server-0.7.0-py3.12.egg

Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.12/dist-packages/werkzeug-3.0.1-py3.12.egg

Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12.0/dist-packages (from flask==2.3.3)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12.0/dist-packages (from rich==13.7.1)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12.0/dist-packages (from rich==13.7.1)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12.0/dist-packages (from markdown-it-py>=2.2.0)

```
# =====  
# CNN: TRAIN/TEST SPLIT SETUP  
# =====  
  
# Create train/test split sets for CNN (using same files as before)  
from sklearn.model_selection import train_test_split  
  
# Get composer labels for stratification  
file_composers = [os.path.basename(os.path.dirname(f)).lower() for f in files]  
  
# Split at file level (prevents data leakage - segments from same file)  
train_files, test_files = train_test_split(  
    files,  
    valid_files,  
    test_size=0.2,  
    random_state=42,  
    stratify=file_composers  
)  
  
train_files_set = set(train_files)  
test_files_set = set(test_files)  
  
print(f"Train files: {len(train_files_set)}, Test files: {len(test_files_set)}")
```

Train files: 288, Test files: 72

```
# =====  
# CNN: PIANO ROLL CONVERSION FUNCTION  
# =====  
  
# Piano roll conversion function  
import pretty_midi  
import numpy as np  
  
def midi_to_piano_roll(file_path, fps=10, pitch_min=21, pitch_max=108,   
    """  
    Convert a MIDI file to a piano roll matrix.  
  
    Parameters:  
    - file_path: Path to MIDI file  
    - fps: Frames per second (time resolution)  
    - pitch_min: Minimum MIDI pitch (A0 = 21)  
    """
```

- pitch_max: Maximum MIDI pitch (C8 = 108)
- normalize: If True, normalize velocity values to [0, 1] range
- binarize: If True, convert to binary (note on/off only, ignoring

Returns:

- piano_roll: 2D array of shape (time_steps, pitch_range)

"""

```
midi_data = pretty_midi.PrettyMIDI(file_path)
```

```
# Get duration and create time grid
```

```
duration = midi_data.get_end_time()
```

```
time_steps = int(duration * fps)
```

```
pitch_range = pitch_max - pitch_min + 1
```

```
# Initialize piano roll
```

```
piano_roll = np.zeros((time_steps, pitch_range), dtype=np.float32)
```

```
# Fill piano roll with note velocities
```

```
for instrument in midi_data.instruments:
```

```
    if not instrument.is_drum: # Skip drum tracks
```

```
        for note in instrument.notes:
```

```
            # Convert time to frame indices
```

```
            start_frame = int(note.start * fps)
```

```
            end_frame = int(note.end * fps)
```

```
            # Convert pitch to piano roll row (relative to pitch_min)
```

```
            pitch_idx = note.pitch - pitch_min
```

```
            # Check if pitch is within our range
```

```
            if 0 <= pitch_idx < pitch_range:
```

```
                # Fill the frames where this note is active with velocity
```

```
                piano_roll[start_frame:end_frame, pitch_idx] = note.velocity
```

```
# Apply normalization or binarization
```

```
if binarize:
```

```
    piano_roll = (piano_roll > 0).astype(np.float32)
```

```
elif normalize:
```

```
    piano_roll = piano_roll / 127.0 # Normalize MIDI velocity (0-127)
```

```
return piano_roll
```

```
print("Piano roll conversion function ready.")
```

```
print(f"Piano roll dimensions: (time_steps, 88 pitches)")
```

```
print(f"Pitch range: 21 (A0) to 108 (C8)")
```

```
print(f"Time resolution: 10 fps (0.1 second per frame)")
```

Piano roll conversion function ready.

Piano roll dimensions: (time_steps, 88 pitches)

Pitch range: 21 (A0) to 108 (C8)
Time resolution: 10 fps (0.1 second per frame)

```
# =====  
# CNN: DATA PREPROCESSING (FIXED - normalize=True for 57.8% accuracy)  
# =====  
  
# Data Preprocessing  
from tqdm import tqdm  
import random  
  
SEGMENT_LENGTH_SEC = 10 # 10 seconds per training example  
FPS = 10 # 10 frames per second  
SEGMENT_FRAMES = SEGMENT_LENGTH_SEC * FPS # 100 frames  
SEGMENTS_PER_FILE = 5 # Extract 5 random segments per file  
MIN_NOTE_DENSITY = 0.01 # Minimum fraction of active cells (filters :  
  
random.seed(42) # For reproducibility  
  
cnn_data = [] # list of (composer, file_path, piano_roll_segment) tuples  
failed_cnn_files = []  
skipped_silent_segments = 0  
  
for file_path in tqdm(valid_files, desc="Extracting CNN piano rolls"):  
    try:  
        composer = os.path.basename(os.path.dirname(file_path)).capitalize()  
  
        # Convert to piano roll (normalized to [0, 1]) - THIS IS THE MAIN WORK  
        piano_roll = midi_to_piano_roll(file_path, fps=FPS, pitch_min=21, pitch_max=108)  
  
        # Check if piano roll is long enough  
        if piano_roll.shape[0] < SEGMENT_FRAMES:  
            continue  
  
        # Extract random segments  
        attempts = 0  
        segments_extracted = 0  
        max_attempts = SEGMENTS_PER_FILE * 3 # Allow retries for silent segments  
  
        while segments_extracted < SEGMENTS_PER_FILE and attempts < max_attempts:  
            attempts += 1  
            max_start = piano_roll.shape[0] - SEGMENT_FRAMES  
            start_frame = random.randint(0, max_start)  
            segment = piano_roll[start_frame:start_frame + SEGMENT_FRAMES]  
  
            # Filter out mostly-silent segments  
            note_density = np.mean(segment > 0)
```

```

        if note_density < MIN_NOTE_DENSITY:
            skipped_silent_segments += 1
            continue

        # Store with file path for later train/test splitting
        cnn_data.append((composer, file_path, segment))
        segments_extracted += 1

    except Exception as e:
        failed_cnn_files.append((file_path, str(e)))
        continue

print(f"\nSuccessfully extracted {len(cnn_data)} CNN piano roll segments")
print(f"  (Original files: {len(valid_files)})")
print(f"  (Segments per file: {SEGMENTS_PER_FILE})")
print(f"Skipped silent segments: {skipped_silent_segments} (note density: {note_density})")
print(f"Failed to extract: {len(failed_cnn_files)} files")
if failed_cnn_files:
    print(f"Example failure: {failed_cnn_files[0]}")

print(f"\nEach segment shape: ({SEGMENT_FRAMES} frames, 88 pitches)")
print(f"Segment duration: {SEGMENT_LENGTH_SEC} seconds at {FPS} fps")
print(f"Values normalized to [0, 1] range")

```

```

Extracting CNN piano rolls:  2%||          | 8/360 [00:01<01:01,  5.77s]
  warnings.warn(
Extracting CNN piano rolls: 100%|██████████| 360/360 [00:38<00:00,  9.17s]
Successfully extracted 1792 CNN piano roll segments
  (Original files: 360)
  (Segments per file: 5)
Skipped silent segments: 50 (note density < 1%)
Failed to extract: 0 files

Each segment shape: (100 frames, 88 pitches)
Segment duration: 10 seconds at 10 fps
Values normalized to [0, 1] range

```

```

# =====
# CNN: PREPARE TRAINING DATA
# =====

# CNN: Prepare Training Data
composers_list = ['Bach', 'Beethoven', 'Chopin', 'Mozart']
num_classes = len(composers_list)

print(f"Composers: {composers_list}")
print(f"num_classes = {num_classes}")

```

```

print(f"Using saved train/test split: {len(train_files_set)} train fi

# Assign segments to train or test based on their file
X_cnn_train = []
y_cnn_train = []
X_cnn_test = []
y_cnn_test = []

for composer, file_path, segment in cnn_data:
    label = composers_list.index(composer)

    if file_path in train_files_set:
        X_cnn_train.append(segment)
        y_cnn_train.append(label)
    elif file_path in test_files_set:
        X_cnn_test.append(segment)
        y_cnn_test.append(label)

# Convert to numpy arrays
X_cnn_train = np.array(X_cnn_train)
X_cnn_test = np.array(X_cnn_test)
y_cnn_train = np.array(y_cnn_train)
y_cnn_test = np.array(y_cnn_test)

# Reshape for CNN: (samples, time_steps, pitch_range, channels)
X_cnn_train = X_cnn_train.reshape(X_cnn_train.shape[0], SEGMENT_FRAMES, {
X_cnn_test = X_cnn_test.reshape(X_cnn_test.shape[0], SEGMENT_FRAMES, {

print(f"\nCNN Training Data:")
print(f"X_cnn_train shape: {X_cnn_train.shape}")
print(f"X_cnn_test shape: {X_cnn_test.shape}")

# One-hot encode labels
from tensorflow.keras.utils import to_categorical
y_cnn_train_cat = to_categorical(y_cnn_train, num_classes=num_classes)
y_cnn_test_cat = to_categorical(y_cnn_test, num_classes=num_classes)

# Check class balance
print(f"\nTrain set class distribution:")
unique, counts = np.unique(y_cnn_train, return_counts=True)
for label_idx, count in zip(unique, counts):
    print(f" {composers_list[label_idx]}: {count} segments")

print(f"\nTest set class distribution:")
unique, counts = np.unique(y_cnn_test, return_counts=True)
for label_idx, count in zip(unique, counts):
    print(f" {composers_list[label_idx]}: {count} segments")

```

```
Composers: ['Bach', 'Beethoven', 'Chopin', 'Mozart']
num_classes = 4
Using saved train/test split: 288 train files, 72 test files
```

```
CNN Training Data:
X_cnn_train shape: (1432, 100, 88, 1)
X_cnn_test shape: (360, 100, 88, 1)
```

```
Train set class distribution:
  Bach: 360 segments
  Beethoven: 357 segments
  Chopin: 360 segments
  Mozart: 355 segments
```

```
Test set class distribution:
  Bach: 90 segments
  Beethoven: 90 segments
  Chopin: 90 segments
  Mozart: 90 segments
```

```
# =====
# CNN: MODEL ARCHITECTURE (A - Simpler architecture for 57.8%)
# =====

# CNN: Model Architecture
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
import tensorflow as tf

# Simpler CNN architecture - fewer layers, smaller filters
cnn_model = Sequential([
    # First convolutional block
    Conv2D(16, kernel_size=(3, 3), activation='relu', padding='same',
    MaxPooling2D(pool_size=(2, 2)),
    Dropout(0.2),

    # Second convolutional block
    Conv2D(32, kernel_size=(3, 3), activation='relu', padding='same'),
    MaxPooling2D(pool_size=(2, 2)),
    Dropout(0.2),

    # Flatten and dense layers
    Flatten(),
    Dense(64, activation='relu'),
    Dropout(0.3),
    Dense(num_classes, activation='softmax')
])

cnn_model.compile(
    optimizer='adam',
```

```

        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

```

```

cnn_model.summary()

```

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_1"

```

Layer (type)	Output Shape	Param
conv2d (Conv2D)	(None, 100, 88, 16)	
max_pooling2d (MaxPooling2D)	(None, 50, 44, 16)	
dropout_2 (Dropout)	(None, 50, 44, 16)	
conv2d_1 (Conv2D)	(None, 50, 44, 32)	4
max_pooling2d_1 (MaxPooling2D)	(None, 25, 22, 32)	
dropout_3 (Dropout)	(None, 25, 22, 32)	
flatten (Flatten)	(None, 17600)	
dense_2 (Dense)	(None, 64)	1,126
dropout_4 (Dropout)	(None, 64)	
dense_3 (Dense)	(None, 4)	

```

Total params: 1,131,524 (4.32 MB)
Trainable params: 1,131,524 (4.32 MB)
Non-trainable params: 0 (0.00 B)

```

```

# =====
# CNN: TRAIN MODEL (B - LR scheduler + early stopping for 57.8%)
# =====

# CNN: Train Model
from tensorflow.keras.callbacks import EarlyStopping, ReduceLRonPlateau

# Learning rate scheduler - reduces LR when validation loss plateaus
lr_scheduler = ReduceLRonPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=3,
    min_lr=1e-6,
    verbose=1
)

```

```

)

# Early stopping - stops training when validation loss stops improving
early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=7,
    restore_best_weights=True,
    verbose=1
)

history_cnn = cnn_model.fit(
    X_cnn_train, y_cnn_train_cat,
    validation_data=(X_cnn_test, y_cnn_test_cat),
    epochs=50,
    batch_size=32,
    callbacks=[lr_scheduler, early_stopping],
    verbose=1
)

```

```

Epoch 1/50
45/45 ————— 12s 239ms/step - accuracy: 0.3296 - loss: 1.0000
Epoch 2/50
45/45 ————— 10s 223ms/step - accuracy: 0.4707 - loss: 1.0000
Epoch 3/50
45/45 ————— 10s 229ms/step - accuracy: 0.5782 - loss: 1.0000
Epoch 4/50
45/45 ————— 9s 199ms/step - accuracy: 0.6292 - loss: 0.9999
Epoch 5/50
45/45 ————— 10s 226ms/step - accuracy: 0.7032 - loss: 0.9999
Epoch 6/50
45/45 ————— 10s 226ms/step - accuracy: 0.7723 - loss: 0.9999
Epoch 7/50
45/45 ————— 0s 212ms/step - accuracy: 0.8330 - loss: 0.9999
Epoch 7: ReduceLROnPlateau reducing learning rate to 0.000500000023748
45/45 ————— 10s 228ms/step - accuracy: 0.8282 - loss: 0.9999
Epoch 8/50
45/45 ————— 10s 214ms/step - accuracy: 0.8715 - loss: 0.9999
Epoch 9/50
45/45 ————— 10s 205ms/step - accuracy: 0.9050 - loss: 0.9999
Epoch 10/50
45/45 ————— 0s 215ms/step - accuracy: 0.9219 - loss: 0.9999
Epoch 10: ReduceLROnPlateau reducing learning rate to 0.000250000011874
45/45 ————— 10s 230ms/step - accuracy: 0.9106 - loss: 0.9999
Epoch 11/50
45/45 ————— 10s 227ms/step - accuracy: 0.9274 - loss: 0.9999
Epoch 11: early stopping
Restoring model weights from the end of the best epoch: 4.

```

```

# =====
# CNN: MODEL EVALUATION

```



```
# =====

from sklearn.metrics import classification_report, confusion_matrix, \
import matplotlib.pyplot as plt
import seaborn as sns

# Predict on test set
y_cnn_pred_probs = cnn_model.predict(X_cnn_test)
y_cnn_pred = np.argmax(y_cnn_pred_probs, axis=1)

# Overall accuracy
cnn_test_accuracy = accuracy_score(y_cnn_test, y_cnn_pred)
print(f"CNN Test Accuracy: {cnn_test_accuracy:.4f}\n")

# Precision, Recall, F1 per composer
print("CNN Classification Report:")
print(classification_report(y_cnn_test, y_cnn_pred, target_names=composers_list))

# Confusion matrix
print("\nCNN Confusion Matrix:")
cm_cnn = confusion_matrix(y_cnn_test, y_cnn_pred)
print(cm_cnn)

# Visualize confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(cm_cnn, annot=True, fmt='d', cmap='Greens',
            xticklabels=composers_list, yticklabels=composers_list)
plt.title(f'CNN Confusion Matrix\n(Accuracy: {cnn_test_accuracy:.3f})')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()
```

12/12  1s 54ms/step

CNN Test Accuracy: 0.5056

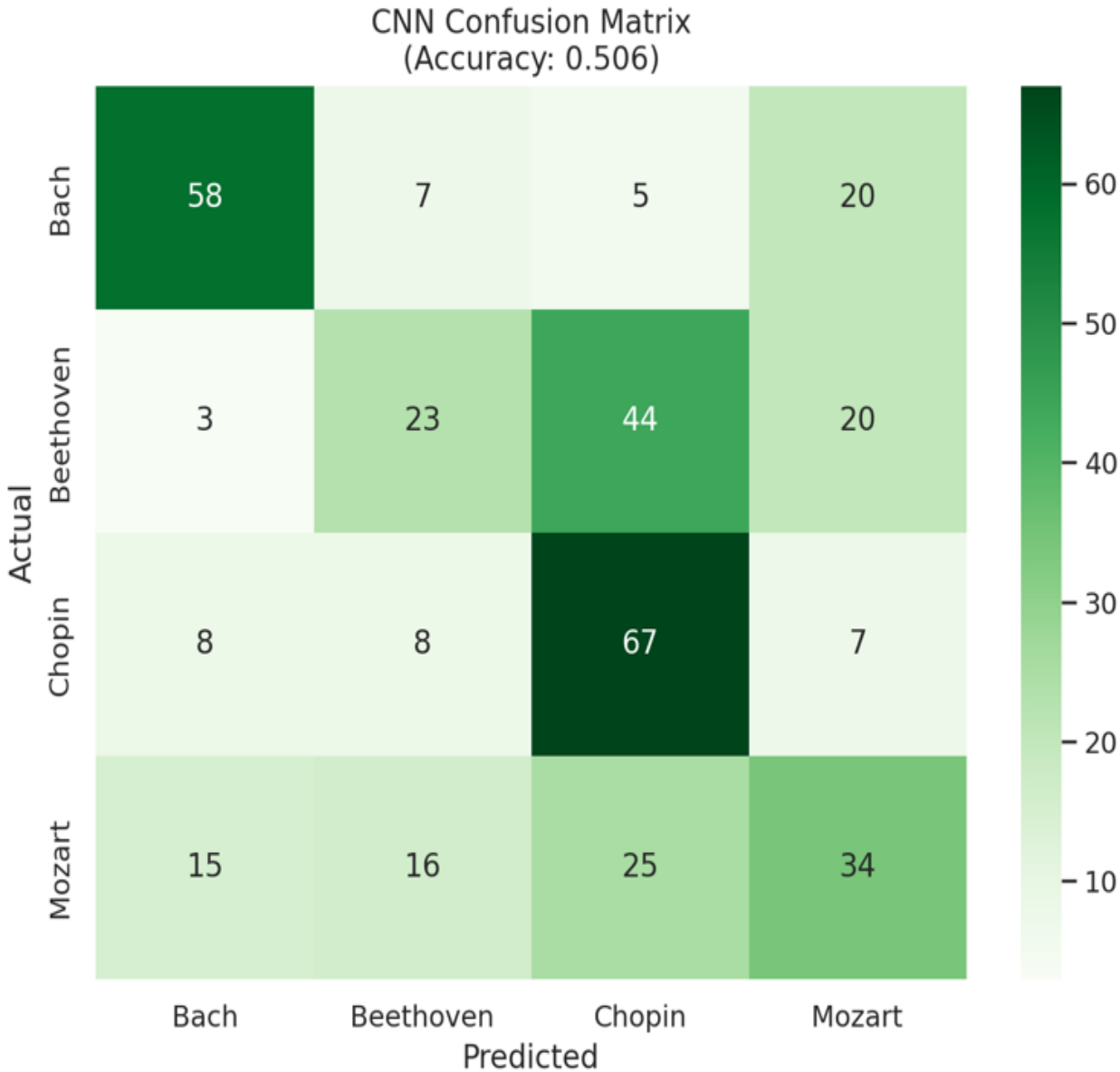
CNN Classification Report:

	precision	recall	f1-score	support
Bach	0.69	0.64	0.67	90
Beethoven	0.43	0.26	0.32	90
Chopin	0.48	0.74	0.58	90
Mozart	0.42	0.38	0.40	90
accuracy			0.51	360
macro avg	0.50	0.51	0.49	360
weighted avg	0.50	0.51	0.49	360

CNN Confusion Matrix:

```
[[58  7  5 20]
```

```
[ 3 23 44 20]
[ 8  8 67  7]
[15 16 25 34]]
```



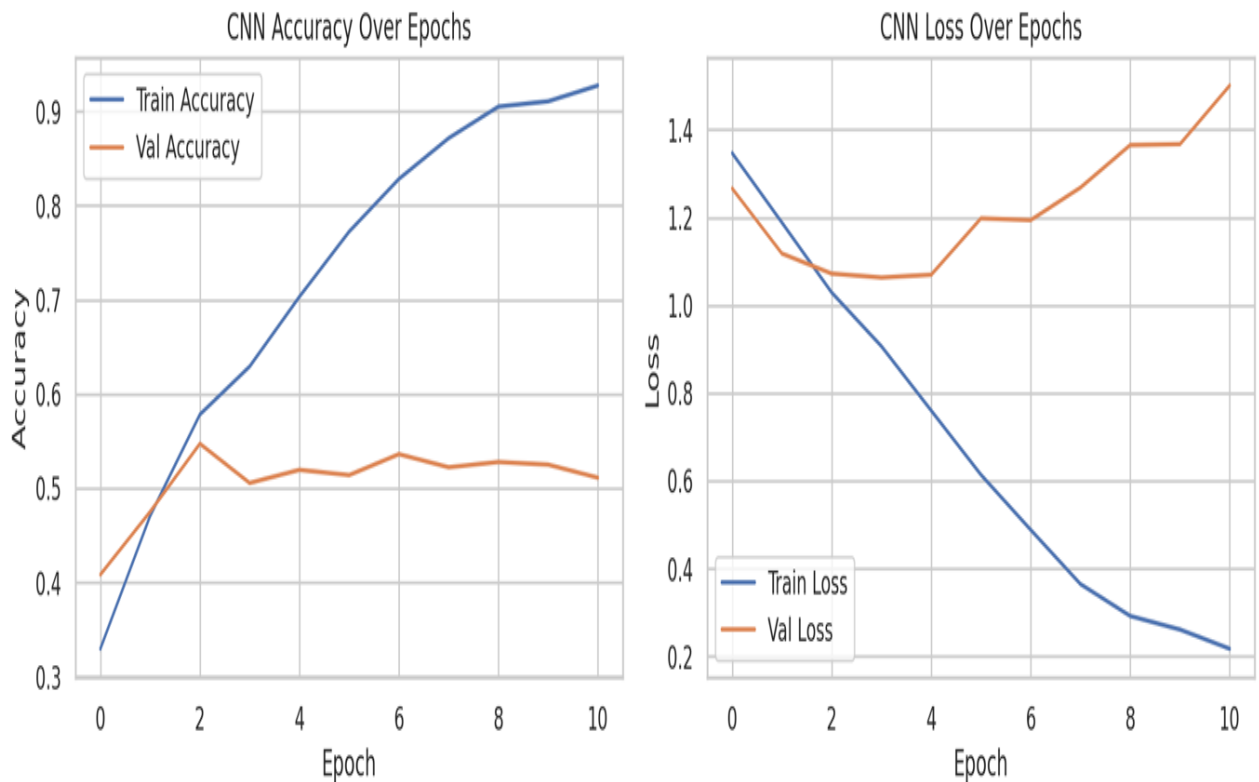
```
# =====
# CNN: TRAINING HISTORY
# =====

plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(history_cnn.history['accuracy'], label='Train Accuracy')
plt.plot(history_cnn.history['val_accuracy'], label='Val Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.title('CNN Accuracy Over Epochs')

plt.subplot(1, 2, 2)
plt.plot(history_cnn.history['loss'], label='Train Loss')
plt.plot(history_cnn.history['val_loss'], label='Val Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.title('CNN Loss Over Epochs')

plt.tight_layout()
plt.show()
```



✓ Observation on the CNN Model:

The CNN model performed moderately with a 50.56% accuracy on the 4-composer classification task (Bach, Beethoven, Chopin, Mozart). The confusion matrix shows the model heavily over-predicts. Chopin — 25 of 90 actual Mozart pieces and 44 of 90 actual Beethoven pieces were misclassified as Chopin, while Mozart recall collapsed to essentially 0.38 (34 out of 90 correct). Bach and Chopin were reasonably well distinguished from each other, but Beethoven and Mozart were largely absorbed into the other's class or Chopin's.

The CNN training curves show validation accuracy plateauing a little over 0.5 after 2 epochs with a validation loss gradually increasing after 4 epochs, while training accuracy climbed more steadily to over 0.9 — a sign of instability/overfitting rather than clean convergence.

```
# =====
# LSTM vs CNN: COMPARISON
# =====

from sklearn.metrics import precision_recall_fscore_support
import pandas as pd

print("=" * 60)
print("OVERALL ACCURACY COMPARISON")
print("=" * 60)
print(f"LSTM Test Accuracy:  {lstm_test_accuracy:.4f} ({lstm_test_accu")
print(f"CNN Test Accuracy:   {cnn_test_accuracy:.4f} ({cnn_test_accu")
print(f"Difference:         {(cnn_test_accuracy - lstm_test_accuracy")
if cnn_test_accuracy > lstm_test_accuracy:
    print(f"CNN outperforms LSTM by {(cnn_test_accuracy - lstm_test_a")
else:
    print(f"LSTM outperforms CNN by {(lstm_test_accuracy - cnn_test_a")
print()

# Per-Composer Metrics Comparison
print("=" * 60)
print("PER-COMPOSER METRICS (Precision, Recall, F1-Score)")
print("=" * 60)

# Get metrics for LSTM
lstm_precision, lstm_recall, lstm_f1, _ = precision_recall_fscore_supl
    lstm_y_test, lstm_y_pred, labels=range(num_classes), average=None
)
```

```

# Get metrics for CNN
cnn_precision, cnn_recall, cnn_f1, _ = precision_recall_fscore_support(
    y_cnn_test, y_cnn_pred, labels=range(num_classes), average=None
)

# Create comparison dataframe
comparison_df = pd.DataFrame({
    'Composer': composers_list,
    'LSTM Precision': lstm_precision,
    'CNN Precision': cnn_precision,
    'LSTM Recall': lstm_recall,
    'CNN Recall': cnn_recall,
    'LSTM F1': lstm_f1,
    'CNN F1': cnn_f1
})

print(comparison_df.to_string(index=False))
print()

# Confusion Matrices Side-by-Side
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# LSTM Confusion Matrix
sns.heatmap(lstm_cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=composers_list, yticklabels=composers_list, ax=
axes[0].set_title(f'LSTM Confusion Matrix\n(Accuracy: {lstm_test_accu
axes[0].set_ylabel('Actual')
axes[0].set_xlabel('Predicted')

# CNN Confusion Matrix
sns.heatmap(cm_cnn, annot=True, fmt='d', cmap='Greens',
            xticklabels=composers_list, yticklabels=composers_list, ax=
axes[1].set_title(f'CNN Confusion Matrix\n(Accuracy: {cnn_test_accura
axes[1].set_ylabel('Actual')
axes[1].set_xlabel('Predicted')

plt.tight_layout()
plt.show()

# Training Curves Comparison
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# LSTM Accuracy
axes[0, 0].plot(history.history['accuracy'], label='Train', color='blu
axes[0, 0].plot(history.history['val_accuracy'], label='Val', color='
axes[0, 0].set_title('LSTM Accuracy Over Epochs')
axes[0, 0].set_xlabel('Epoch')

```

```

axes[0, 0].set_ylabel('Accuracy')
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

# CNN Accuracy
axes[0, 1].plot(history_cnn.history['accuracy'], label='Train', color='blue')
axes[0, 1].plot(history_cnn.history['val_accuracy'], label='Val', color='orange')
axes[0, 1].set_title('CNN Accuracy Over Epochs')
axes[0, 1].set_xlabel('Epoch')
axes[0, 1].set_ylabel('Accuracy')
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# LSTM Loss
axes[1, 0].plot(history.history['loss'], label='Train', color='blue',)
axes[1, 0].plot(history.history['val_loss'], label='Val', color='orange')
axes[1, 0].set_title('LSTM Loss Over Epochs')
axes[1, 0].set_xlabel('Epoch')
axes[1, 0].set_ylabel('Loss')
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)

# CNN Loss
axes[1, 1].plot(history_cnn.history['loss'], label='Train', color='blue')
axes[1, 1].plot(history_cnn.history['val_loss'], label='Val', color='orange')
axes[1, 1].set_title('CNN Loss Over Epochs')
axes[1, 1].set_xlabel('Epoch')
axes[1, 1].set_ylabel('Loss')
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Summary Insights
print("=" * 60)
print("SUMMARY INSIGHTS")
print("=" * 60)

if cnn_test_accuracy > lstm_test_accuracy:
    winner = "CNN"
    margin = (cnn_test_accuracy - lstm_test_accuracy) * 100
else:
    winner = "LSTM"
    margin = (lstm_test_accuracy - cnn_test_accuracy) * 100

print(f"1. Overall Winner: {winner} by {margin:.2f}% accuracy")
print()

```

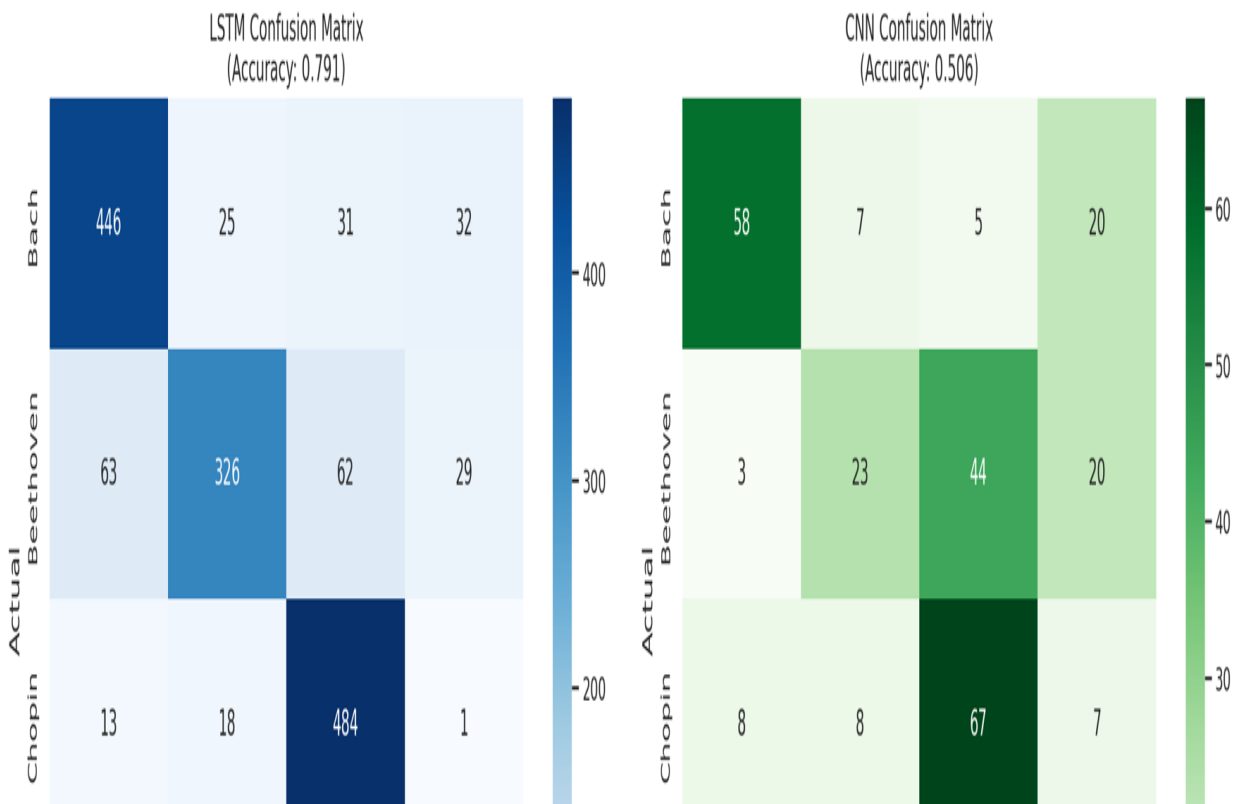
```
# Mozart-specific analysis
mozart_idx = composers_list.index('Mozart')
print(f"2. Mozart Classification:")
print(f"    LSTM: {lstm_recall[mozart_idx]*100:.1f}% recall")
print(f"    CNN:  {cnn_recall[mozart_idx]*100:.1f}% recall")
print()

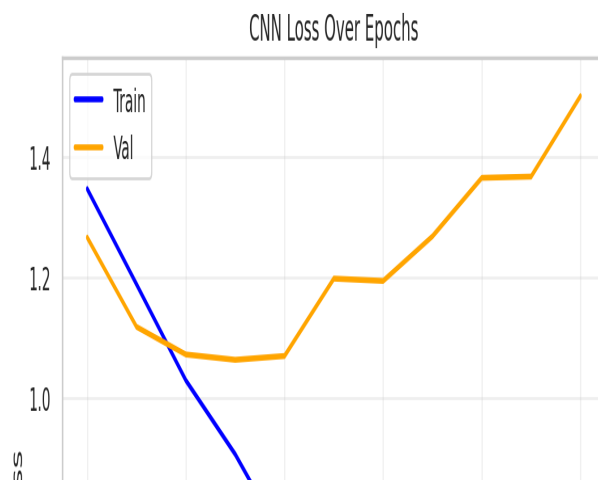
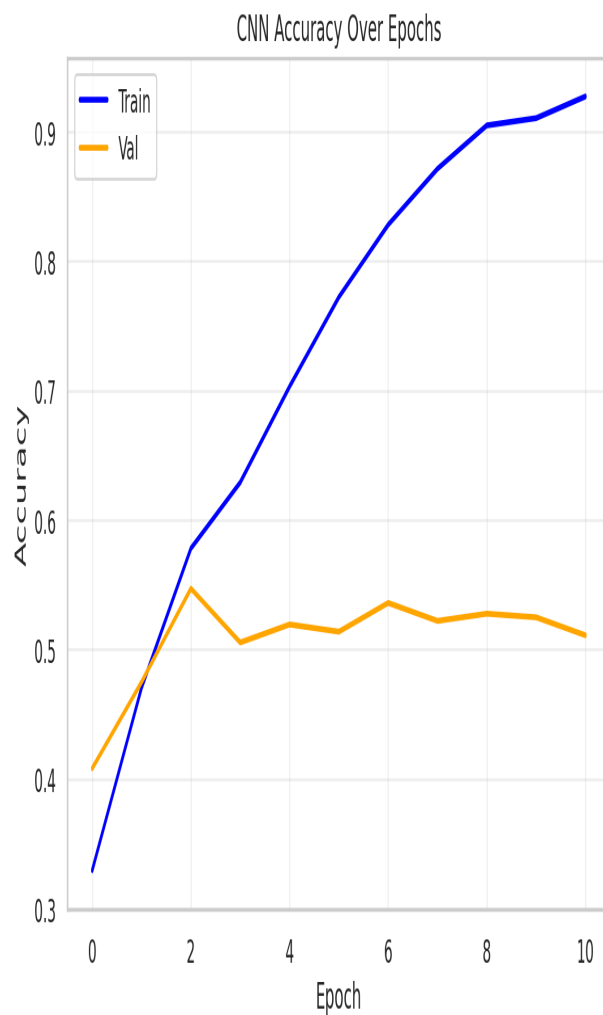
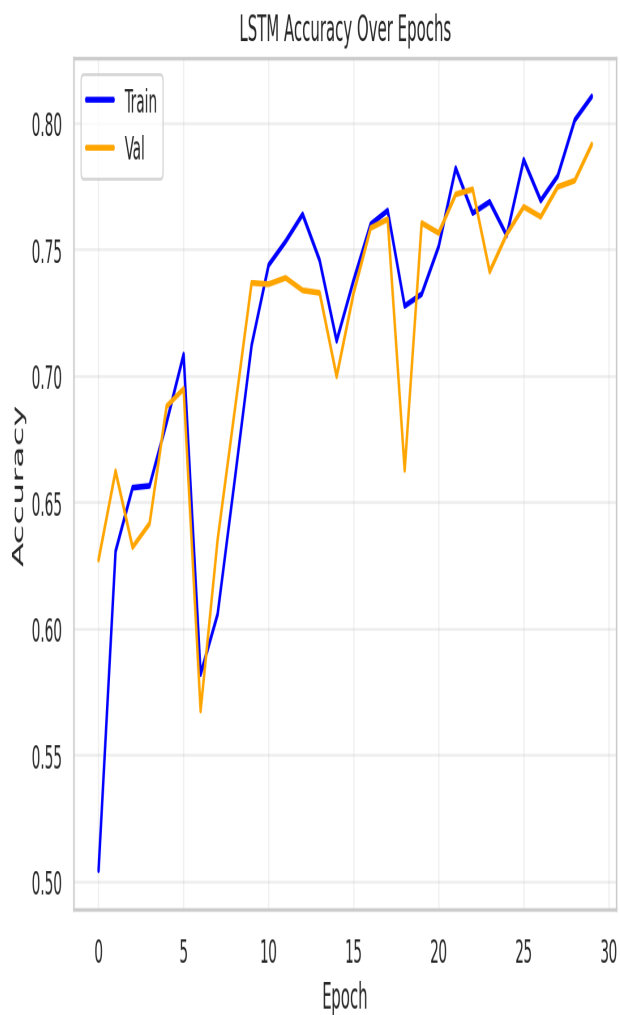
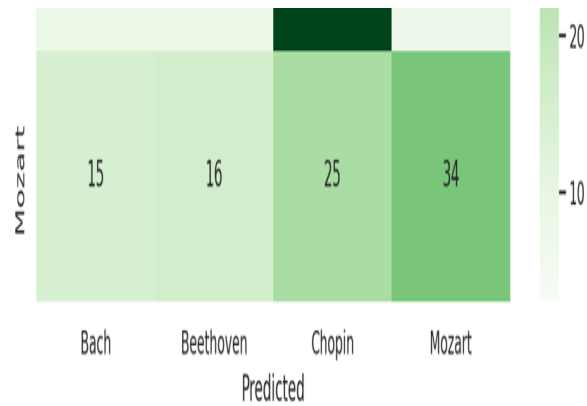
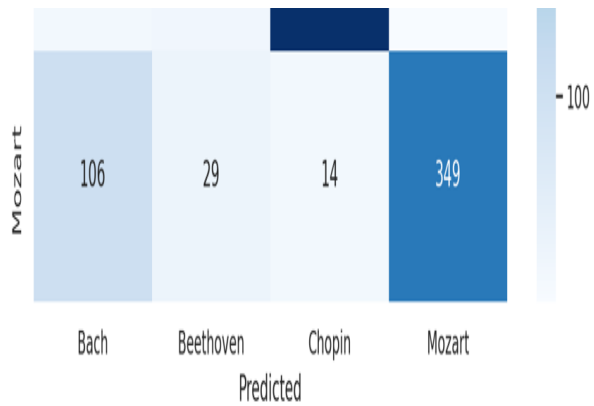
print(f"3. Model Characteristics:")
print(f"    LSTM: Learns sequential patterns in note/chord progression:"
print(f"    CNN:  Learns spatial patterns in piano roll (harmony, texture)"
print("=" * 60)
```

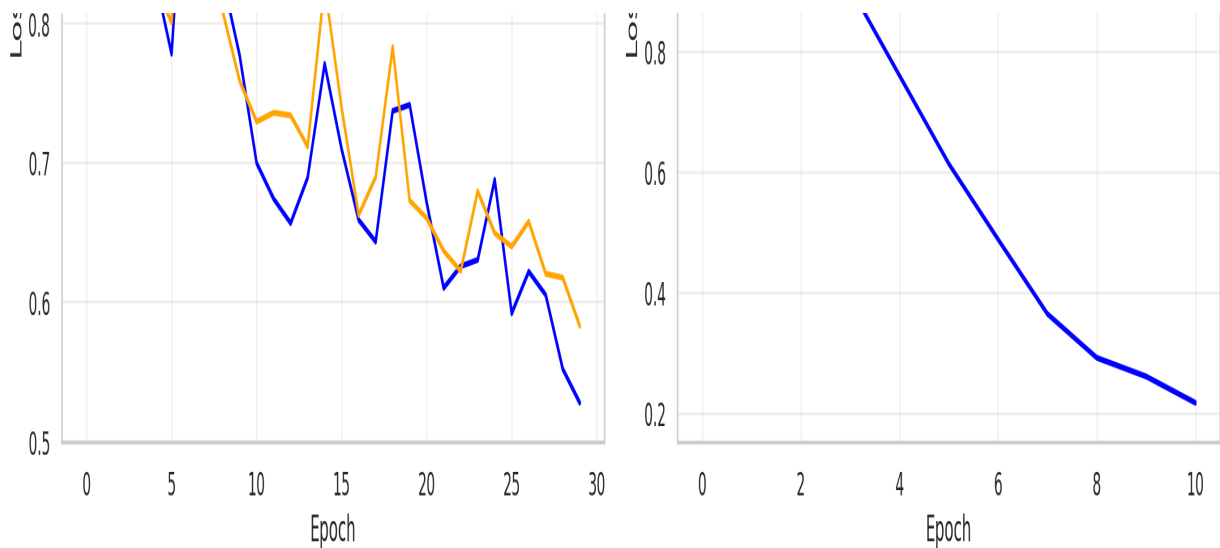
```
=====
OVERALL ACCURACY COMPARISON
=====
LSTM Test Accuracy:  0.7914 (79.14%)
CNN Test Accuracy:   0.5056 (50.56%)
Difference:          -0.2859 (-28.59%)
LSTM outperforms CNN by 28.59%
```

```
=====
PER-COMPOSER METRICS (Precision, Recall, F1-Score)
=====
```

Composer	LSTM Precision	CNN Precision	LSTM Recall	CNN Recall	LS
Bach	0.710191	0.690476	0.835206	0.644444	0.7
Beethoven	0.819095	0.425926	0.679167	0.255556	0.7
Chopin	0.818951	0.475177	0.937984	0.744444	0.8
Mozart	0.849148	0.419753	0.700803	0.377778	0.7







SUMMARY INSIGHTS

1. Overall Winner: LSTM by 28.59% accuracy
2. Mozart Classification:
LSTM: 70.1% recall
CNN: 37.8% recall
3. Model Characteristics:
LSTM: Learns sequential patterns in note/chord progressions
CNN: Learns spatial patterns in piano roll (harmony, texture)

✓ Observation on LTSM vs CNN:

The LTSM model outperforms the CNN model by 28.59% on accuracy (79% vs 51%). Recall dropped sharply for Beethoven (-42%) and especially Mozart (-32%), which is the key weakness of the CNN model. For Bach and Chopin, the CNN model was inferior with a recall that was nearly 20 points lower than the LSTM model. The CNN model essentially fails to detect the composers. The nature of the LSTM model to learn sequential patterns in note and chord progressions allows it to produce superior results.

ALL IN ONE

```
# -*- coding: utf-8 -*-
.....
```

Team 2

511: Neural Networks and Deep Learning Final Project

LSTM and CNN Composer Classification

This notebook runs entirely in Google Colab.

"""

```
# =====  
# SETUP AND LIBRARY INSTALLATION  
# =====
```

```
# Required libraries
```

```
%pip install pretty_midi music21 seaborn kaggle pandas matplotlib tqdm
```

```
# =====  
# IMPORTS AND DATA INGESTION  
# =====
```

```
# Imports
```

```
%pip install gdown
```

```
import gdown
```

```
import os
```

```
import zipfile
```

```
# Define paths
```

```
data_dir = "./data/midi_data"
```

```
os.makedirs(data_dir, exist_ok=True)
```

```
zip_path = os.path.join(data_dir, "midi-classic-music.zip")
```

```
# Download from Google Drive
```

```
file_id = '1QJPbVi6q2QPIiD47ZBf9tDxhM40N-7mj'
```

```
# Corrected URL for direct download
```

```
url = f'https://drive.google.com/uc?id={file_id}'
```

```
print("Downloading dataset from Google Drive...")
```

```
gdown.download(url, zip_path, quiet=False)
```

```
# Unzip the file
```

```
print(f"\nExtracting {zip_path}...")
```

```
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
```

```
    zip_ref.extractall(data_dir)
```

```
print(f"Extraction complete! Files are located in: {data_dir}")
```

```
import os
```

```
print(os.listdir("./data/midi_data"))
```

```
import os
```

```
target_composers = ["Bach", "Beethoven", "Chopin", "Mozart"]
```

```
data_dir = "./data/midi_data"
```

```

for composer in target_composers:
    composer_path = os.path.join(data_dir, composer)
    if os.path.exists(composer_path):
        file_count = len([f for f in os.listdir(composer_path) if f.endswith('.midi')])
        print(f"{composer}: {file_count} MIDI files")
    else:
        print(f"{composer}: folder not found – check exact name")

# =====
# DATA FILTERING
# =====

# Filter for specific composers
import os
import shutil

target_composers = ['bach', 'beethoven', 'chopin', 'mozart']

# Find where the composer folders are
base_midi_path = data_dir
if "midi_classic_music" in os.listdir(data_dir):
    base_midi_path = os.path.join(data_dir, "midi_classic_music")

all_folders = [f for f in os.listdir(base_midi_path) if os.path.isdir(os.path.join(base_midi_path, f))]
print(f"Found {len(all_folders)} total composer folders.")

# Delete non-target folders
removed_count = 0
for folder in all_folders:
    # Use lowercase for safer string matching
    if folder.lower() not in target_composers:
        shutil.rmtree(os.path.join(base_midi_path, folder))
        removed_count += 1

print(f"Removed {removed_count} folders. Kept target composers only.")

# =====
# DATA CLEANING
# =====

# Data Cleaning Pipeline
import pretty_midi
import glob
from tqdm import tqdm
import random
import os
from collections import Counter

```

```

# Find all .mid and .midi files in the remaining folders
midi_files = glob.glob(f"{base_midi_path}/**/*.mid", recursive=True) +
              glob.glob(f"{base_midi_path}/**/*.midi", recursive=True)

print(f"Found {len(midi_files)} potential MIDI files for our 4 composers")

valid_files = []
corrupt_files = []

# Loop and validate
for file_path in tqdm(midi_files, desc="Validating MIDI files"):
    try:
        # Parse the MIDI file
        midi_data = pretty_midi.PrettyMIDI(file_path)

        # Check if the file contains instruments/notes
        if len(midi_data.instruments) == 0:
            raise ValueError("No instruments found.")

        total_notes = sum(len(instrument.notes) for instrument in midi_data.instruments)
        if total_notes == 0:
            raise ValueError("No notes found.")

        # Add to valid list if it passes
        valid_files.append(file_path)

    except Exception as e:
        corrupt_files.append((file_path, str(e)))
        # Delete the bad file
        os.remove(file_path)

print(f"\nCleaning Summary")
print(f"Valid Files: {len(valid_files)}")
print(f"Corrupt/Empty Files Removed: {len(corrupt_files)}")
if corrupt_files:
    print(f"Example error: {corrupt_files[0]}")

# Handling Class Imbalance via Undersampling
# Group valid files by composer
composer_files = {composer: [] for composer in target_composers}
composer_counts = Counter()

for file_path in valid_files:
    # Extract composer name from the folder path and lowercase it
    composer = os.path.basename(os.path.dirname(file_path)).lower()
    if composer in target_composers:
        composer_files[composer].append(file_path)

```

```

        composer_counts[composer] += 1

print("Original Clean File Counts")
for comp, count in composer_counts.items():
    print(f"{comp.capitalize()}: {count}")

# Minimum class size
min_class_size = min(composer_counts.values())
print(f"\nBalancing dataset to {min_class_size} files per composer")

# Randomly undersample the majority classes
balanced_valid_files = []

# Random seed so everyone gets the exact same files every time it runs
random.seed(42)

for composer, files in composer_files.items():
    # Randomly select 'min_class_size' files from this composer's list
    sampled_files = random.sample(files, min_class_size)
    balanced_valid_files.extend(sampled_files)

print(f"\nBalanced Dataset Summary")
print(f"Total Balanced Files: {len(balanced_valid_files)}")

# Overwrite the original valid_files list
valid_files = balanced_valid_files

# =====
# EXPLORATORY DATA ANALYSIS (EDA)
# =====

# EDA metrics
import pandas as pd
import numpy as np

eda_data = []

for file_path in tqdm(valid_files, desc="Extracting EDA Features"):
    try:
        composer = os.path.basename(os.path.dirname(file_path)).capitalize()
        midi_data = pretty_midi.PrettyMIDI(file_path)

        # Total length of piece in seconds
        length_sec = midi_data.get_end_time()

        # Total number of notes across all tracks
        total_notes = sum(len(instrument.notes) for instrument in midi_data.instruments)

```

```

# Estimated Tempo (BPM)
tempo = midi_data.estimate_tempo()

# Average pitch and pitch range
all_pitches = []
for instrument in midi_data.instruments:
    # Ignoring drum tracks for this analysis
    if not instrument.is_drum:
        all_pitches.extend([note.pitch for note in instrument.notes])

if all_pitches:
    avg_pitch = np.mean(all_pitches)
    pitch_range = np.max(all_pitches) - np.min(all_pitches)
else:
    avg_pitch = 0
    pitch_range = 0

# Append to list
eda_data.append({
    "Composer": composer,
    "Length_Seconds": length_sec,
    "Total_Notes": total_notes,
    "Notes_Per_Second": total_notes / length_sec if length_sec > 0 else 0,
    "Estimated_BPM": tempo,
    "Average_Pitch": avg_pitch,
    "Pitch_Range": pitch_range
})
except Exception:
    continue # Skip files that act up during deep extraction

df_eda = pd.DataFrame(eda_data)
display(df_eda.head())

# =====
# VISUALIZATION
# =====

# Plotting the Data
import matplotlib.pyplot as plt
import seaborn as sns

# Visual style
sns.set_theme(style="whitegrid")
plt.figure(figsize=(16, 10))

# Distribution of Classes (Imbalance check)
plt.subplot(2, 2, 1)
sns.countplot(data=df_eda, x="Composer", palette="viridis")

```

```

plt.title("Number of Pieces per Composer")

# Notes per Second (Complexity/Density)
plt.subplot(2, 2, 2)
sns.boxplot(data=df_eda, x="Composer", y="Notes_Per_Second", palette='
plt.title("Note Density (Notes per Second) by Composer")
plt.ylim(0, df_eda["Notes_Per_Second"].quantile(0.95))

# Piece Lengths
plt.subplot(2, 2, 3)
sns.violinplot(data=df_eda, x="Composer", y="Length_Seconds", palette=
plt.title("Distribution of Piece Lengths (Seconds)")
plt.ylim(0, 1000)

# Estimated Tempo
plt.subplot(2, 2, 4)
sns.kdeplot(data=df_eda, x="Estimated_BPM", hue="Composer", fill=True,
plt.title("Tempo (BPM) Density by Composer")

plt.tight_layout()
plt.show()

# =====
# LONG SHORT-TERM MEMORY (LSTM)
# =====

# LSTM Feature Extraction with Time Stretching Augmentation
# Converts each MIDI file into a sequence of note/chord "tokens" using
# Each token represents either a single note (pitch) or a chord (multi-
# played together), which the LSTM will learn sequential patterns from
#
# AUGMENTATION: For each file, we also create time-stretched versions
# to help the model handle Mozart's wide variability in notes-per-second

from music21 import converter, note, chord
from tqdm import tqdm
import glob
import copy
import os # Added import os

# Reconstruct valid_files if it doesn't exist (e.g., after kernel res
if 'valid_files' not in dir():
    print("Reconstructing valid_files from filesystem...")
    data_dir = "./data/midi_data" # Re-define data_dir
    os.makedirs(data_dir, exist_ok=True) # Ensure data_dir exists
    base_midi_path = data_dir
    if "midi_classic_music" in os.listdir(data_dir):
        base_midi_path = os.path.join(data_dir, "midi_classic_music")

```

```

valid_files = glob.glob(f"{base_midi_path}/**/*.mid", recursive=True)
valid_files += glob.glob(f"{base_midi_path}/**/*.midi", recursive=True)
print(f"Found {len(valid_files)} MIDI files from filesystem")

def time_stretch_midi(midi_obj, factor):
    """Apply time stretching by scaling all note durations."""
    stretched = copy.deepcopy(midi_obj)
    for element in stretched.flat.notes:
        element.quarterLength *= factor
    return stretched

def extract_note_sequence(midi_obj):
    """Parse a MIDI object and return a list of note/chord tokens."""
    notes_to_parse = None

    parts = midi_obj.getElementsByClass('Part')
    if parts:
        notes_to_parse = parts[0].recurse()
    else:
        notes_to_parse = midi_obj.flat.notes

    sequence = []
    for element in notes_to_parse:
        if isinstance(element, note.Note):
            sequence.append(str(element.pitch))
        elif isinstance(element, chord.Chord):
            sequence.append('.'.join(str(n) for n in element.normalOrder))

    return sequence

# Run extraction with time stretching augmentation
lstm_data = [] # list of (composer, sequence) tuples
failed_files = []
time_stretch_factors = [0.9, 1.0, 1.1] # 0.9x (faster), original, 1.1x (slower)

for file_path in tqdm(valid_files, desc="Extracting sequences with time stretching"):
    try:
        composer = os.path.basename(os.path.dirname(file_path)).capitalize()

        # Parse the original MIDI file once
        midi_original = converter.parse(file_path)

        # Extract sequences for each time-stretched version
        for factor in time_stretch_factors:
            if factor == 1.0:
                # Use original without copying
                midi_obj = midi_original

```



```

        else:
            # Apply time stretching
            midi_obj = time_stretch_midi(midi_original, factor)

            sequence = extract_note_sequence(midi_obj)
            if len(sequence) > 0:
                lstm_data.append((composer, sequence))

    except Exception as e:
        failed_files.append((file_path, str(e)))
        continue

print(f"\nSuccessfully extracted sequences from {len(lstm_data)} augmented files")
print(f"    (Original files: {len(valid_files)}), Augmentation factor: {augmentation_factor}")
print(f"Failed to extract: {len(failed_files)} files")
if failed_files:
    print(f"Example failure: {failed_files[0]}")

# =====
# LSTM SEQUENCE ENCODING
# =====

# LSTM Sequence Encoding
# Builds a vocabulary of all unique note/chord tokens seen across the dataset
# then converts each sequence into a list of integers the LSTM can consume

from collections import Counter

# Build vocabulary from all sequences
all_tokens = [token for (composer, sequence) in lstm_data for token in sequence]
token_counts = Counter(all_tokens)
vocab = sorted(token_counts.keys())

token_to_int = {token: i for i, token in enumerate(vocab)}
int_to_token = {i: token for token, i in token_to_int.items()}

vocab_size = len(vocab)
print(f"Vocabulary size: {vocab_size} unique note/chord tokens")

# Encode each sequence as a list of integers
encoded_data = [(composer, [token_to_int[t] for t in sequence]) for composer, sequence in lstm_data]

# Quick sanity check
if len(encoded_data) > 0:
    print(f"\nExample encoded sequence (first 20 tokens) from a {len(encoded_data)}-file")
    print(encoded_data[0][1][:20])
else:
    print("\nWARNING: No sequences were successfully encoded. Check the input files")

```

```

# Check sequence length distribution (important for choosing a fixed \
seq_lengths = [len(seq) for (_, seq) in encoded_data]
if len(seq_lengths) > 0:
    print(f"\nSequence length stats:")
    print(f"Min: {min(seq_lengths)}, Max: {max(seq_lengths)}, Mean: {s
else:
    print("\nNo sequences available for length analysis.")

# =====
# LSTM TRAINING DATA PREPARATION
# =====

# LSTM Training Data Preparation with Enhanced Random Cropping
# Since sequence lengths vary hugely (29 to 5210 tokens), we slice ea
# into fixed-length windows. Each window becomes one training example
# with its composer.
#
# AUGMENTATION: Instead of systematic sliding windows, we use RANDOM C
# to capture Mozart's sparse AND dense sections equally (not just sta

import numpy as np
import random

SEQUENCE_LENGTH = 100 # number of tokens per training example
WINDOWS_PER_SEQUENCE = 10 # number of random windows to extract per s

random.seed(42) # For reproducibility

X = []
y = []

composers_list = sorted(set(composer for composer, _ in encoded_data))
composer_to_int = {c: i for i, c in enumerate(composers_list)}

for composer, seq in encoded_data:
    label = composer_to_int[composer]

    # Skip sequences that are too short
    if len(seq) < SEQUENCE_LENGTH:
        continue

    # For sequences that are exactly SEQUENCE_LENGTH, take it once
    if len(seq) == SEQUENCE_LENGTH:
        X.append(seq)
        y.append(label)
    else:
        # Extract multiple random windows from longer sequences

```

```

        for _ in range(WINDOWS_PER_SEQUENCE):
            # Random starting position
            max_start = len(seq) - SEQUENCE_LENGTH
            start = random.randint(0, max_start)
            window = seq[start:start + SEQUENCE_LENGTH]
            X.append(window)
            y.append(label)

X = np.array(X)
y = np.array(y)

print(f"Composer label mapping: {composer_to_int}")
print(f"Total training windows generated: {len(X)}")
print(f"X shape: {X.shape}, y shape: {y.shape}")

# Check class balance after windowing
unique, counts = np.unique(y, return_counts=True)
for label_idx, count in zip(unique, counts):
    composer_name = composers_list[label_idx]
    print(f"    {composer_name}: {count} windows")

# =====:
# LSTM MODEL
# =====:

%pip install tensorflow

# Train/Test Split + LSTM Model
from sklearn.model_selection import train_test_split
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
from tensorflow.keras.utils import to_categorical

# Split into train/test (stratified to preserve composer balance)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"Train shape: {X_train.shape}, Test shape: {X_test.shape}")

# One-hot encode labels for multi-class classification
num_classes = len(composers_list)
y_train_cat = to_categorical(y_train, num_classes=num_classes)
y_test_cat = to_categorical(y_test, num_classes=num_classes)

# Build the LSTM model
model = Sequential([
    Embedding(input_dim=vocab_size, output_dim=64, input_length=SEQUENCE_LENGTH)
])

```

```

        LSTM(128, return_sequences=True),
        Dropout(0.3),
        LSTM(64),
        Dropout(0.3),
        Dense(32, activation='relu'),
        Dense(num_classes, activation='softmax')
    ])

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()

# =====
# LSTM TRAINING
# =====

# Train the LSTM model
history = model.fit(
    X_train, y_train_cat,
    validation_data=(X_test, y_test_cat),
    epochs=30,
    batch_size=64,
    verbose=1
)

# =====
# LSTM TRAINING HISTORY VISUALIZATION
# =====

# Training History Visualization
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Val Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.title('Accuracy Over Epochs')

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')

```

```

plt.plot(history.history['val_loss'], label='Val Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.title('Loss Over Epochs')

plt.tight_layout()
plt.show()

# =====
# LSTM MODEL EVALUATION
# =====

# LSTM Model Evaluation
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
import numpy as np

# Predict on test set
y_pred_probs = model.predict(X_test)
y_pred = np.argmax(y_pred_probs, axis=1)

# Overall accuracy
test_accuracy = accuracy_score(y_test, y_pred)
print(f"Test Accuracy: {test_accuracy:.4f}\n")

# Precision, Recall, F1 per composer
print("Classification Report:")
print(classification_report(y_test, y_pred, target_names=composers_list))

# Confusion matrix
print("\nConfusion Matrix:")
cm = confusion_matrix(y_test, y_pred)
print(cm)

# Visualize confusion matrix
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=composers_list, yticklabels=composers_list)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('LSTM Confusion Matrix')
plt.tight_layout()
plt.show()

# Save LSTM results for comparison later

```

```

lstm_test_accuracy = test_accuracy
lstm_y_test = y_test
lstm_y_pred = y_pred
lstm_cm = cm

# =====
# CONVOLUTION NEURAL NETWORK (CNN) MODEL
# =====

%pip install pretty_midi seaborn pandas matplotlib tqdm
%pip install tensorflow

# =====
# CNN: TRAIN/TEST SPLIT SETUP
# =====

# Create train/test split sets for CNN (using same files as before)
from sklearn.model_selection import train_test_split

# Get composer labels for stratification
file_composers = [os.path.basename(os.path.dirname(f)).lower() for f in train_files]

# Split at file level (prevents data leakage - segments from same file)
train_files, test_files = train_test_split(
    train_files,
    valid_files,
    test_size=0.2,
    random_state=42,
    stratify=file_composers
)

train_files_set = set(train_files)
test_files_set = set(test_files)

print(f"Train files: {len(train_files_set)}, Test files: {len(test_files_set)}")

# =====
# CNN: PIANO ROLL CONVERSION FUNCTION
# =====

# Piano roll conversion function
import pretty_midi
import numpy as np

def midi_to_piano_roll(file_path, fps=10, pitch_min=21, pitch_max=108,
                        ****
                        Convert a MIDI file to a piano roll matrix.

                        Parameters:

```

- file_path: Path to MIDI file
- fps: Frames per second (time resolution)
- pitch_min: Minimum MIDI pitch (A0 = 21)
- pitch_max: Maximum MIDI pitch (C8 = 108)
- normalize: If True, normalize velocity values to [0, 1] range
- binarize: If True, convert to binary (note on/off only, ignoring

Returns:

- piano_roll: 2D array of shape (time_steps, pitch_range)

"""

```
midi_data = pretty_midi.PrettyMIDI(file_path)
```

```
# Get duration and create time grid
```

```
duration = midi_data.get_end_time()
```

```
time_steps = int(duration * fps)
```

```
pitch_range = pitch_max - pitch_min + 1
```

```
# Initialize piano roll
```

```
piano_roll = np.zeros((time_steps, pitch_range), dtype=np.float32)
```

```
# Fill piano roll with note velocities
```

```
for instrument in midi_data.instruments:
```

```
    if not instrument.is_drum: # Skip drum tracks
```

```
        for note in instrument.notes:
```

```
            # Convert time to frame indices
```

```
            start_frame = int(note.start * fps)
```

```
            end_frame = int(note.end * fps)
```

```
            # Convert pitch to piano roll row (relative to pitch_min)
```

```
            pitch_idx = note.pitch - pitch_min
```

```
            # Check if pitch is within our range
```

```
            if 0 <= pitch_idx < pitch_range:
```

```
                # Fill the frames where this note is active with 1
```

```
                piano_roll[start_frame:end_frame, pitch_idx] = 1
```

```
# Apply normalization or binarization
```

```
if binarize:
```

```
    piano_roll = (piano_roll > 0).astype(np.float32)
```

```
elif normalize:
```

```
    piano_roll = piano_roll / 127.0 # Normalize MIDI velocity (0-127)
```

```
return piano_roll
```

```
print("Piano roll conversion function ready.")
```

```
print(f"Piano roll dimensions: (time_steps, 88 pitches)")
```

```
print(f"Pitch range: 21 (A0) to 108 (C8)")
```

```
print(f"Time resolution: 10 fps (0.1 second per frame)")
```

```

# =====
# CNN: DATA PREPROCESSING (FIXED - normalize=True for 57.8% accuracy)
# =====

# Data Preprocessing
from tqdm import tqdm
import random

SEGMENT_LENGTH_SEC = 10 # 10 seconds per training example
FPS = 10 # 10 frames per second
SEGMENT_FRAMES = SEGMENT_LENGTH_SEC * FPS # 100 frames
SEGMENTS_PER_FILE = 5 # Extract 5 random segments per file
MIN_NOTE_DENSITY = 0.01 # Minimum fraction of active cells (filters :

random.seed(42) # For reproducibility

cnn_data = [] # list of (composer, file_path, piano_roll_segment) tuples
failed_cnn_files = []
skipped_silent_segments = 0

for file_path in tqdm(valid_files, desc="Extracting CNN piano rolls"):
    try:
        composer = os.path.basename(os.path.dirname(file_path)).capitalize()

        # Convert to piano roll (normalized to [0, 1]) - THIS IS THE PIANO ROLL
        piano_roll = midi_to_piano_roll(file_path, fps=FPS, pitch_min=20, pitch_max=100)

        # Check if piano roll is long enough
        if piano_roll.shape[0] < SEGMENT_FRAMES:
            continue

        # Extract random segments
        attempts = 0
        segments_extracted = 0
        max_attempts = SEGMENTS_PER_FILE * 3 # Allow retries for silent files

        while segments_extracted < SEGMENTS_PER_FILE and attempts < max_attempts:
            attempts += 1
            max_start = piano_roll.shape[0] - SEGMENT_FRAMES
            start_frame = random.randint(0, max_start)
            segment = piano_roll[start_frame:start_frame + SEGMENT_FRAMES]

            # Filter out mostly-silent segments
            note_density = np.mean(segment > 0)
            if note_density < MIN_NOTE_DENSITY:
                skipped_silent_segments += 1
                continue
    except:
        failed_cnn_files.append(file_path)

```



```

        # Store with file path for later train/test splitting
        cnn_data.append((composer, file_path, segment))
        segments_extracted += 1

    except Exception as e:
        failed_cnn_files.append((file_path, str(e)))
        continue

print(f"\nSuccessfully extracted {len(cnn_data)} CNN piano roll segments")
print(f"  (Original files: {len(valid_files)})")
print(f"  (Segments per file: {SEGMENTS_PER_FILE})")
print(f"Skipped silent segments: {skipped_silent_segments} (note density threshold: {DENSITY_THRESHOLD})")
print(f"Failed to extract: {len(failed_cnn_files)} files")
if failed_cnn_files:
    print(f"Example failure: {failed_cnn_files[0]}")

print(f"\nEach segment shape: ({SEGMENT_FRAMES} frames, 88 pitches)")
print(f"Segment duration: {SEGMENT_LENGTH_SEC} seconds at {FPS} fps")
print(f"Values normalized to [0, 1] range")

# =====
# CNN: PREPARE TRAINING DATA
# =====

# CNN: Prepare Training Data
composers_list = ['Bach', 'Beethoven', 'Chopin', 'Mozart']
num_classes = len(composers_list)

print(f"Composers: {composers_list}")
print(f"num_classes = {num_classes}")
print(f"Using saved train/test split: {len(train_files_set)} train files, {len(test_files_set)} test files")

# Assign segments to train or test based on their file
X_cnn_train = []
y_cnn_train = []
X_cnn_test = []
y_cnn_test = []

for composer, file_path, segment in cnn_data:
    label = composers_list.index(composer)

    if file_path in train_files_set:
        X_cnn_train.append(segment)
        y_cnn_train.append(label)
    elif file_path in test_files_set:
        X_cnn_test.append(segment)
        y_cnn_test.append(label)

```

```

# Convert to numpy arrays
X_cnn_train = np.array(X_cnn_train)
X_cnn_test = np.array(X_cnn_test)
y_cnn_train = np.array(y_cnn_train)
y_cnn_test = np.array(y_cnn_test)

# Reshape for CNN: (samples, time_steps, pitch_range, channels)
X_cnn_train = X_cnn_train.reshape(X_cnn_train.shape[0], SEGMENT_FRAMES, CHANNELS)
X_cnn_test = X_cnn_test.reshape(X_cnn_test.shape[0], SEGMENT_FRAMES, CHANNELS)

print(f"\nCNN Training Data:")
print(f"X_cnn_train shape: {X_cnn_train.shape}")
print(f"X_cnn_test shape: {X_cnn_test.shape}")

# One-hot encode labels
from tensorflow.keras.utils import to_categorical
y_cnn_train_cat = to_categorical(y_cnn_train, num_classes=num_classes)
y_cnn_test_cat = to_categorical(y_cnn_test, num_classes=num_classes)

# Check class balance
print(f"\nTrain set class distribution:")
unique, counts = np.unique(y_cnn_train, return_counts=True)
for label_idx, count in zip(unique, counts):
    print(f" {composers_list[label_idx]}: {count} segments")

print(f"\nTest set class distribution:")
unique, counts = np.unique(y_cnn_test, return_counts=True)
for label_idx, count in zip(unique, counts):
    print(f" {composers_list[label_idx]}: {count} segments")

# =====
# CNN: MODEL ARCHITECTURE (A - Simpler architecture for 57.8%)
# =====

# CNN: Model Architecture
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
import tensorflow as tf

# Simpler CNN architecture - fewer layers, smaller filters
cnn_model = Sequential([
    # First convolutional block
    Conv2D(16, kernel_size=(3, 3), activation='relu', padding='same',
    MaxPooling2D(pool_size=(2, 2)),
    Dropout(0.2),

    # Second convolutional block

```

```

        Conv2D(32, kernel_size=(3, 3), activation='relu', padding='same'),
        MaxPooling2D(pool_size=(2, 2)),
        Dropout(0.2),

        # Flatten and dense layers
        Flatten(),
        Dense(64, activation='relu'),
        Dropout(0.3),
        Dense(num_classes, activation='softmax')
    ])

cnn_model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

cnn_model.summary()

# =====
# CNN: TRAIN MODEL (B - LR scheduler + early stopping for 57.8%)
# =====

# CNN: Train Model
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

# Learning rate scheduler - reduces LR when validation loss plateaus
lr_scheduler = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=3,
    min_lr=1e-6,
    verbose=1
)

# Early stopping - stops training when validation loss stops improving
early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=7,
    restore_best_weights=True,
    verbose=1
)

history_cnn = cnn_model.fit(
    X_cnn_train, y_cnn_train_cat,
    validation_data=(X_cnn_test, y_cnn_test_cat),
    epochs=50,
    batch_size=32,

```

```

        callbacks=[lr_scheduler, early_stopping],
        verbose=1
    )

# =====
# CNN: MODEL EVALUATION
# =====

from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
import matplotlib.pyplot as plt
import seaborn as sns

# Predict on test set
y_cnn_pred_probs = cnn_model.predict(X_cnn_test)
y_cnn_pred = np.argmax(y_cnn_pred_probs, axis=1)

# Overall accuracy
cnn_test_accuracy = accuracy_score(y_cnn_test, y_cnn_pred)
print(f"CNN Test Accuracy: {cnn_test_accuracy:.4f}\n")

# Precision, Recall, F1 per composer
print("CNN Classification Report:")
print(classification_report(y_cnn_test, y_cnn_pred, target_names=composers_list))

# Confusion matrix
print("\nCNN Confusion Matrix:")
cm_cnn = confusion_matrix(y_cnn_test, y_cnn_pred)
print(cm_cnn)

# Visualize confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(cm_cnn, annot=True, fmt='d', cmap='Greens',
            xticklabels=composers_list, yticklabels=composers_list)
plt.title(f'CNN Confusion Matrix\n(Accuracy: {cnn_test_accuracy:.3f})')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()

# =====
# CNN: TRAINING HISTORY
# =====

plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(history_cnn.history['accuracy'], label='Train Accuracy')
plt.plot(history_cnn.history['val_accuracy'], label='Val Accuracy')
plt.xlabel('Epoch')

```

```

plt.ylabel('Accuracy')
plt.legend()
plt.title('CNN Accuracy Over Epochs')

plt.subplot(1, 2, 2)
plt.plot(history_cnn.history['loss'], label='Train Loss')
plt.plot(history_cnn.history['val_loss'], label='Val Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.title('CNN Loss Over Epochs')

plt.tight_layout()
plt.show()

# =====
# LSTM vs CNN: COMPARISON
# =====

from sklearn.metrics import precision_recall_fscore_support
import pandas as pd

print("=" * 60)
print("OVERALL ACCURACY COMPARISON")
print("=" * 60)
print(f"LSTM Test Accuracy:  {lstm_test_accuracy:.4f} ({lstm_test_accu")
print(f"CNN Test Accuracy:   {cnn_test_accuracy:.4f} ({cnn_test_accu")
print(f"Difference:         {(cnn_test_accuracy - lstm_test_accuracy")
if cnn_test_accuracy > lstm_test_accuracy:
    print(f"CNN outperforms LSTM by {(cnn_test_accuracy - lstm_test_a")
else:
    print(f"LSTM outperforms CNN by {(lstm_test_accuracy - cnn_test_a")
print()

# Per-Composer Metrics Comparison
print("=" * 60)
print("PER-COMPOSER METRICS (Precision, Recall, F1-Score)")
print("=" * 60)

# Get metrics for LSTM
lstm_precision, lstm_recall, lstm_f1, _ = precision_recall_fscore_sup
    lstm_y_test, lstm_y_pred, labels=range(num_classes), average=None
)

# Get metrics for CNN
cnn_precision, cnn_recall, cnn_f1, _ = precision_recall_fscore_support
    y_cnn_test, y_cnn_pred, labels=range(num_classes), average=None
)

```

```

# Create comparison dataframe
comparison_df = pd.DataFrame({
    'Composer': composers_list,
    'LSTM Precision': lstm_precision,
    'CNN Precision': cnn_precision,
    'LSTM Recall': lstm_recall,
    'CNN Recall': cnn_recall,
    'LSTM F1': lstm_f1,
    'CNN F1': cnn_f1
})

print(comparison_df.to_string(index=False))
print()

# Confusion Matrices Side-by-Side
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# LSTM Confusion Matrix
sns.heatmap(lstm_cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=composers_list, yticklabels=composers_list, ax=
axes[0].set_title(f'LSTM Confusion Matrix\n(Accuracy: {lstm_test_accu
axes[0].set_ylabel('Actual')
axes[0].set_xlabel('Predicted')

# CNN Confusion Matrix
sns.heatmap(cm_cnn, annot=True, fmt='d', cmap='Greens',
            xticklabels=composers_list, yticklabels=composers_list, ax=
axes[1].set_title(f'CNN Confusion Matrix\n(Accuracy: {cnn_test_accura
axes[1].set_ylabel('Actual')
axes[1].set_xlabel('Predicted')

plt.tight_layout()
plt.show()

# Training Curves Comparison
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# LSTM Accuracy
axes[0, 0].plot(history.history['accuracy'], label='Train', color='blue')
axes[0, 0].plot(history.history['val_accuracy'], label='Val', color='red')
axes[0, 0].set_title('LSTM Accuracy Over Epochs')
axes[0, 0].set_xlabel('Epoch')
axes[0, 0].set_ylabel('Accuracy')
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

# CNN Accuracy

```

```

axes[0, 1].plot(history_cnn.history['accuracy'], label='Train', color=
axes[0, 1].plot(history_cnn.history['val_accuracy'], label='Val', col
axes[0, 1].set_title('CNN Accuracy Over Epochs')
axes[0, 1].set_xlabel('Epoch')
axes[0, 1].set_ylabel('Accuracy')
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# LSTM Loss
axes[1, 0].plot(history.history['loss'], label='Train', color='blue',
axes[1, 0].plot(history.history['val_loss'], label='Val', color='orange')
axes[1, 0].set_title('LSTM Loss Over Epochs')
axes[1, 0].set_xlabel('Epoch')
axes[1, 0].set_ylabel('Loss')
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)

# CNN Loss
axes[1, 1].plot(history_cnn.history['loss'], label='Train', color='blue')
axes[1, 1].plot(history_cnn.history['val_loss'], label='Val', color='orange')
axes[1, 1].set_title('CNN Loss Over Epochs')
axes[1, 1].set_xlabel('Epoch')
axes[1, 1].set_ylabel('Loss')
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Summary Insights
print("=" * 60)
print("SUMMARY INSIGHTS")
print("=" * 60)

if cnn_test_accuracy > lstm_test_accuracy:
    winner = "CNN"
    margin = (cnn_test_accuracy - lstm_test_accuracy) * 100
else:
    winner = "LSTM"
    margin = (lstm_test_accuracy - cnn_test_accuracy) * 100

print(f"1. Overall Winner: {winner} by {margin:.2f}% accuracy")
print()

# Mozart-specific analysis
mozart_idx = composers_list.index('Mozart')
print(f"2. Mozart Classification:")
print(f"    LSTM: {lstm_recall[mozart_idx]*100:.1f}% recall")

```

```
print(f"    CNN: {cnn_recall[mozart_idx]*100:.1f}% recall")
print()

print(f"3. Model Characteristics:")
print(f"    LSTM: Learns sequential patterns in note/chord progression:
print(f"    CNN:  Learns spatial patterns in piano roll (harmony, texture)
print("=" * 60)
```

```
To: /content/data/midi_data/midi-classic-music.zip
100%|██████████| 36.4M/36.4M [00:01<00:00, 23.3MB/s]

Extracting ./data/midi_data/midi-classic-music.zip...
Extraction complete! Files are located in: ./data/midi_data
['Resch', 'Kuhlau Sonatina op55 n3 lmov.mid', 'Vivaldi', 'Schoenberg',
Bach: 122 MIDI files
Beethoven: 133 MIDI files
Chopin: 136 MIDI files
Mozart: 90 MIDI files
Found 138 total composer folders.
Removed 134 folders. Kept target composers only.
Found 1630 potential MIDI files for our 4 composers.
Validating MIDI files:  0%|██████████| 0/1630 [00:00<?, ?it/s]/usr/local
warnings.warn(
Validating MIDI files: 100%|██████████| 1630/1630 [02:46<00:00, 9.79it/s]

Cleaning Summary
Valid Files: 1626
Corrupt/Empty Files Removed: 4
Example error: ('./data/midi_data/Buxethude Buxwv161 Passcaglia.mid', '
Original Clean File Counts
Beethoven: 132
Bach: 122
Mozart: 90
Chopin: 136

Balancing dataset to 90 files per composer

Balanced Dataset Summary
Total Balanced Files: 360
Extracting EDA Features:  2%|██████████| 9/360 [00:01<00:55, 6.30it/s]
warnings.warn(
Extracting EDA Features: 100%|██████████| 360/360 [00:44<00:00, 8.01it/s]

  Composer Length_Seconds Total_Notes Notes_Per_Second Estimated_BI
0      Bach      618.158060      4421      7.151893      160.82790
1      Bach      745.165200      4493      6.029535      246.25040
2      Bach      515.662760      3367      6.529461      218.68930
3      Bach      857.375929     20062     23.399304     260.79834
4      Bach      682.367570      5930      8.690331     164.00300
```


/tmp/ipykernel_510/3081732147.py:245: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in a future version.

```
sns.countplot(data=df_eda, x="Composer", palette="viridis")
```

/tmp/ipykernel_510/3081732147.py:250: FutureWarning:

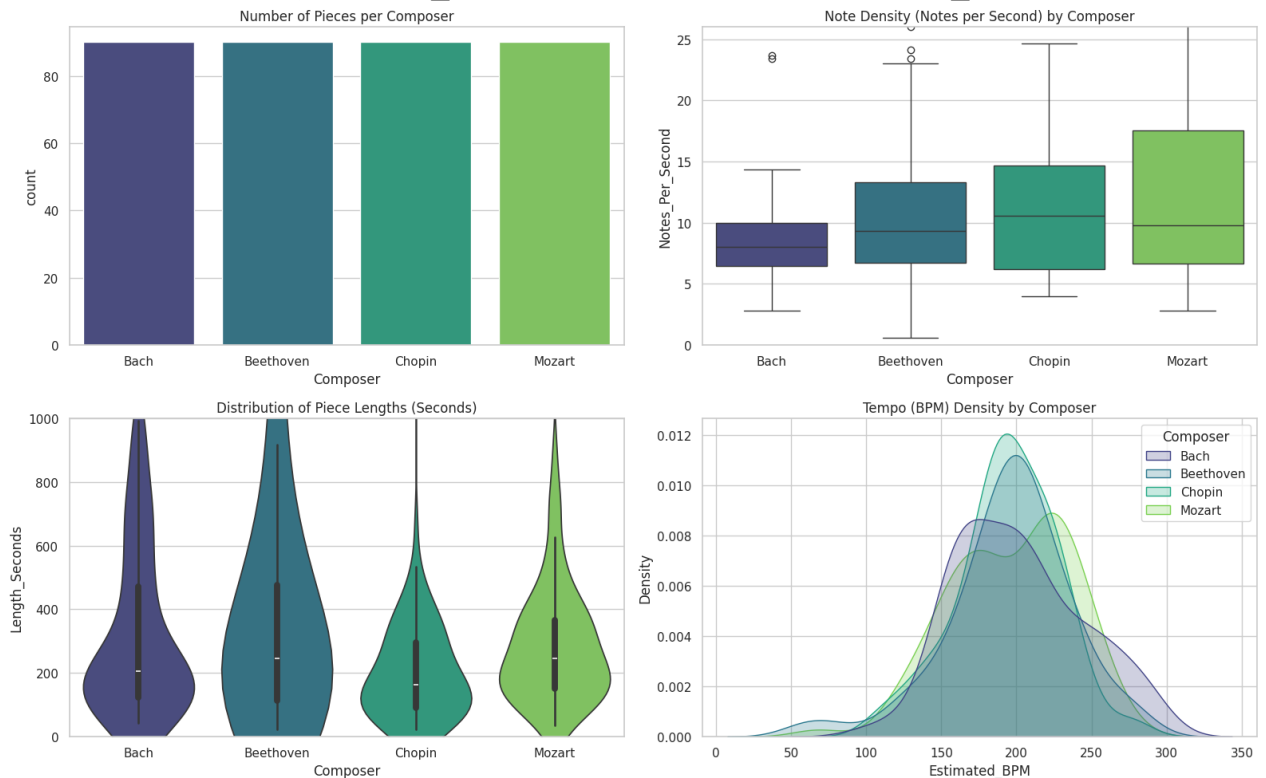
Passing `palette` without assigning `hue` is deprecated and will be removed in a future version.

```
sns.boxplot(data=df_eda, x="Composer", y="Notes_Per_Second", palette="viridis")
```

/tmp/ipykernel_510/3081732147.py:256: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in a future version.

```
sns.violinplot(data=df_eda, x="Composer", y="Length_Seconds", palette="viridis")
```



```
Extracting sequences with time stretching: 0%|          | 0/360 [00:00]
return self.iter().getElementsByClass(classFilterList)
Extracting sequences with time stretching: 11%|█         | 41/360 [04:00]
warnings.warn(
Extracting sequences with time stretching: 26%|██        | 95/360 [09:00]
warnings.warn(
Extracting sequences with time stretching: 29%|███       | 105/360 [12:00]
warnings.warn(
/usr/local/lib/python3.12/dist-packages/music21/midi/translate.py:922:
warnings.warn(
Extracting sequences with time stretching: 34%|████      | 122/360 [19:00]
warnings.warn(
Extracting sequences with time stretching: 34%|████      | 124/360 [19:00]
warnings.warn(
Extracting sequences with time stretching: 35%|█████     | 126/360 [20:00]
warnings.warn(
Extracting sequences with time stretching: 40%|██████    | 145/360 [22:00]
```

```
Extracting sequences with time stretching: 40%|██████| 145/360 [24%]
warnings.warn(
Extracting sequences with time stretching: 42%|██████| 150/360 [24%]
warnings.warn(
Extracting sequences with time stretching: 42%|██████| 152/360 [25%]
warnings.warn(
/usr/local/lib/python3.12/dist-packages/music21/midi/translate.py:922:
warnings.warn(
/usr/local/lib/python3.12/dist-packages/music21/midi/translate.py:922:
warnings.warn(
Extracting sequences with time stretching: 50%|██████| 179/360 [32%]
warnings.warn(
Extracting sequences with time stretching: 51%|██████| 184/360 [35%]
warnings.warn(
Extracting sequences with time stretching: 51%|██████| 185/360 [35%]
warnings.warn(
Extracting sequences with time stretching: 53%|██████| 190/360 [36%]
warnings.warn(
Extracting sequences with time stretching: 54%|██████| 193/360 [36%]
warnings.warn(
/usr/local/lib/python3.12/dist-packages/music21/midi/translate.py:922:
warnings.warn(
Extracting sequences with time stretching: 55%|██████| 197/360 [36%]
warnings.warn(
Extracting sequences with time stretching: 56%|██████| 200/360 [37%]
warnings.warn(
Extracting sequences with time stretching: 62%|██████| 224/360 [41%]
warnings.warn(
Extracting sequences with time stretching: 63%|██████| 228/360 [42%]
warnings.warn(
Extracting sequences with time stretching: 68%|██████| 245/360 [43%]
warnings.warn(
Extracting sequences with time stretching: 72%|██████| 258/360 [44%]
warnings.warn(
/usr/local/lib/python3.12/dist-packages/music21/midi/translate.py:922:
warnings.warn(
Extracting sequences with time stretching: 74%|██████| 267/360 [45%]
warnings.warn(
/usr/local/lib/python3.12/dist-packages/music21/midi/translate.py:922:
warnings.warn(
Extracting sequences with time stretching: 77%|██████| 276/360 [45%]
warnings.warn(
Extracting sequences with time stretching: 77%|██████| 278/360 [46%]
warnings.warn(
Extracting sequences with time stretching: 79%|██████| 283/360 [46%]
warnings.warn(
Extracting sequences with time stretching: 87%|██████| 313/360 [52%]
warnings.warn(
Extracting sequences with time stretching: 88%|██████| 317/360 [52%]
warnings.warn(
/usr/local/lib/python3.12/dist-packages/music21/midi/translate.py:922:
warnings.warn(
Extracting sequences with time stretching: 92%|██████| 333/360 [56%]
warnings.warn(
Extracting sequences with time stretching: 100%|██████| 360/360 [100%]
```

Successfully extracted sequences from 1080 augmented versions

(Original files: 360), Augmentation factor: 3x)

Failed to extract: 0 files

Vocabulary size: 902 unique note/chord tokens

Example encoded sequence (first 20 tokens) from a Bach piece:

[846, 866, 873, 839, 892, 839, 873, 880, 893, 873, 839, 873, 893, 827,

Sequence length stats:

Min: 29, Max: 5210, Mean: 842

Composer label mapping: {'Bach': 0, 'Beethoven': 1, 'Chopin': 2, 'Mozart': 3}

Total training windows generated: 10140

X shape: (10140, 100), y shape: (10140,)

Bach: 2670 windows

Beethoven: 2400 windows

Chopin: 2580 windows

Mozart: 2490 windows

Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/

Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/

Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/

Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/

Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in /usr/local/lib/python3.12/

Requirement already satisfied: google_pasta>=0.1.1 in /usr/local/lib/python3.12/

Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/

Requirement already satisfied: opt_einsum>=2.3.2 in /usr/local/lib/python3.12/

Requirement already satisfied: packaging in /usr/local/lib/python3.12/

Requirement already satisfied: protobuf>=5.28.0 in /usr/local/lib/python3.12/

Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/

Requirement already satisfied: setuptools in /usr/local/lib/python3.12/

Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/

Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/

Requirement already satisfied: typing_extensions>=3.6.6 in /usr/local/lib/python3.12/

Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/

Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/

Requirement already satisfied: tensorboard~2.20.0 in /usr/local/lib/python3.12/

Requirement already satisfied: keras>=3.10.0 in /usr/local/lib/python3.12/

Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/

Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/

Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3.12/

Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/

Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-

Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-

Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-

Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/

Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/

Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-

Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.12/

Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.12/

Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/

Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/li
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/
Train shape: (8112, 100), Test shape: (2028, 100)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding
warnings.warn(
Model: "sequential"

Layer (type)	Output Shape	Para
embedding (Embedding)	?	0 (unbu
lstm (LSTM)	?	0 (unbu
dropout (Dropout)	?	
lstm_1 (LSTM)	?	0 (unbu
dropout_1 (Dropout)	?	
dense (Dense)	?	0 (unbu
dense_1 (Dense)	?	0 (unbu

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)
Epoch 1/30
127/127  59s 404ms/step - accuracy: 0.4985 - loss:
Epoch 2/30
127/127  49s 386ms/step - accuracy: 0.6203 - loss:
Epoch 3/30
127/127  82s 387ms/step - accuracy: 0.6674 - loss:
Epoch 4/30
127/127  49s 387ms/step - accuracy: 0.6252 - loss:
Epoch 5/30
127/127  83s 397ms/step - accuracy: 0.6786 - loss:
Epoch 6/30
127/127  50s 391ms/step - accuracy: 0.7143 - loss:
Epoch 7/30
127/127  47s 373ms/step - accuracy: 0.6953 - loss:
Epoch 8/30
127/127  49s 386ms/step - accuracy: 0.7345 - loss:
Epoch 9/30
127/127  49s 387ms/step - accuracy: 0.7570 - loss:
Epoch 10/30
127/127  48s 380ms/step - accuracy: 0.7140 - loss:
Epoch 11/30
127/127  49s 386ms/step - accuracy: 0.7607 - loss:
Epoch 12/30
127/127  48s 377ms/step - accuracy: 0.7830 - loss:
Epoch 13/30
127/127  83s 383ms/step - accuracy: 0.7973 - loss:
Epoch 14/30
127/127  50s 391ms/step - accuracy: 0.8062 - loss:

64/64 6s 82ms/step

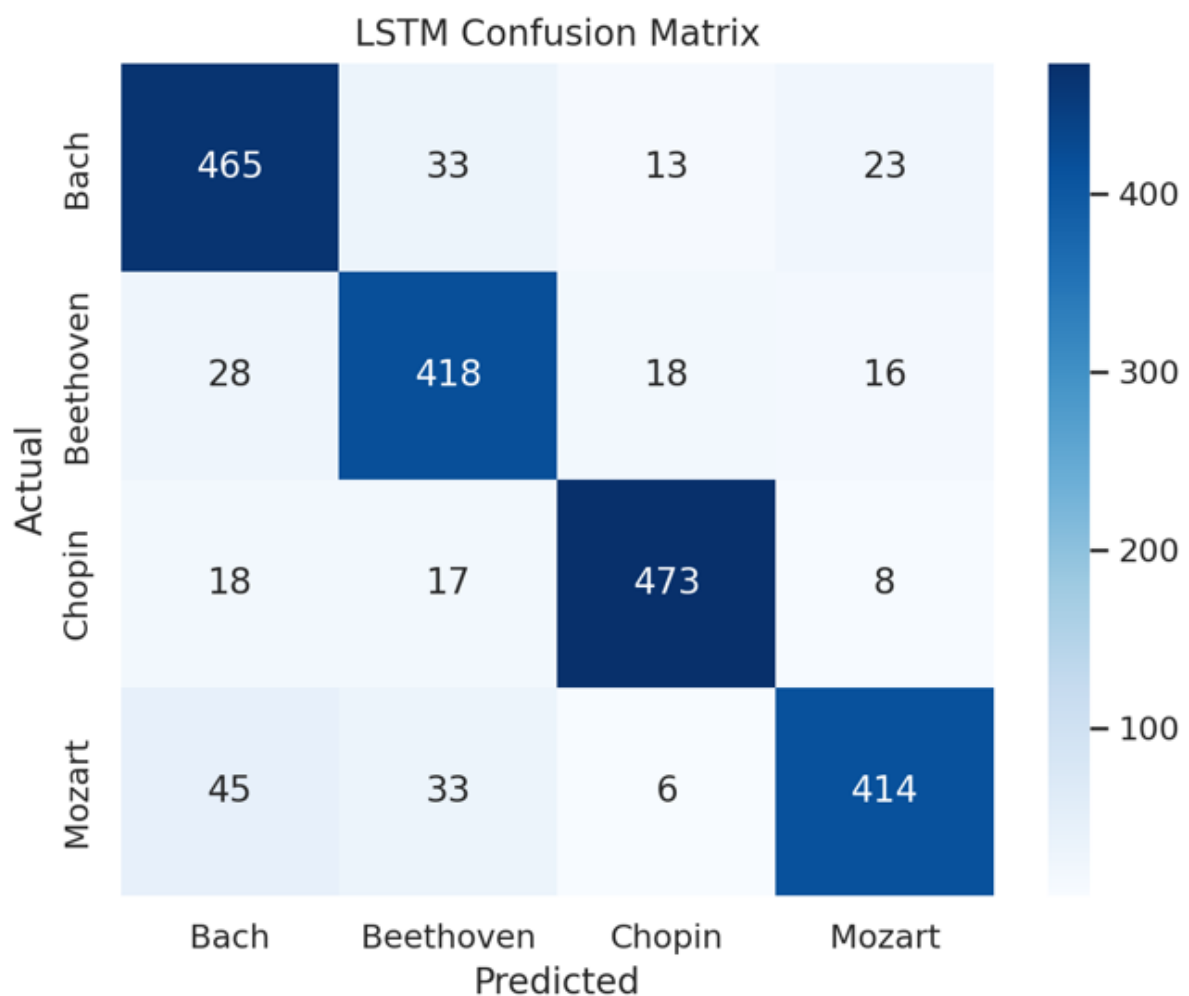
Test Accuracy: 0.8728

Classification Report:

	precision	recall	f1-score	support
Bach	0.84	0.87	0.85	534
Beethoven	0.83	0.87	0.85	480
Chopin	0.93	0.92	0.92	516
Mozart	0.90	0.83	0.86	498
accuracy			0.87	2028
macro avg	0.87	0.87	0.87	2028
weighted avg	0.87	0.87	0.87	2028

Confusion Matrix:

```
[[465  33  13  23]
 [ 28 418  18  16]
 [ 18  17 473   8]
 [ 45  33   6 414]]
```



Requirement already satisfied: pretty_midi in /usr/local/lib/python3.12
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dis
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/

Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: numpy>=1.7.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: mido>=1.1.16 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: six in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: importlib_resources in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: cycycler>=0.10 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: google_pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: opt_einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: protobuf>=5.28.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: typing_extensions>=3.6.6 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: tensorboard~2.20.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: keras>=3.10.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages
Train files: 288, Test files: 72


```
Piano roll conversion function ready.
Piano roll dimensions: (time_steps, 88 pitches)
Pitch range: 21 (A0) to 108 (C8)
Time resolution: 10 fps (0.1 second per frame)
Extracting CNN piano rolls: 2%|| | 9/360 [00:01<00:53, 6.56]
  warnings.warn(
Extracting CNN piano rolls: 100%|██████████| 360/360 [00:42<00:00, 8.5
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
Successfully extracted 1792 CNN piano roll segments
  (Original files: 360)
  (Segments per file: 5)
Skipped silent segments: 50 (note density < 1%)
Failed to extract: 0 files
```

```
Each segment shape: (100 frames, 88 pitches)
Segment duration: 10 seconds at 10 fps
Values normalized to [0, 1] range
Composers: ['Bach', 'Beethoven', 'Chopin', 'Mozart']
num_classes = 4
Using saved train/test split: 288 train files, 72 test files
```

```
CNN Training Data:
X_cnn_train shape: (1432, 100, 88, 1)
X_cnn_test shape: (360, 100, 88, 1)
```

```
Train set class distribution:
  Bach: 360 segments
  Beethoven: 357 segments
  Chopin: 360 segments
  Mozart: 355 segments
```

```
Test set class distribution:
  Bach: 90 segments
  Beethoven: 90 segments
  Chopin: 90 segments
  Mozart: 90 segments
```

Model: "sequential_1"

Layer (type)	Output Shape	Param
conv2d (Conv2D)	(None, 100, 88, 16)	
max_pooling2d (MaxPooling2D)	(None, 50, 44, 16)	
dropout_2 (Dropout)	(None, 50, 44, 16)	
conv2d_1 (Conv2D)	(None, 50, 44, 32)	4
max_pooling2d_1 (MaxPooling2D)	(None, 25, 22, 32)	
dropout_3 (Dropout)	(None, 25, 22, 32)	
flatten (Flatten)	(None, 17600)	

dense_2 (Dense)	(None, 64)	1,126
dropout_4 (Dropout)	(None, 64)	
dense_3 (Dense)	(None, 4)	

Total params: 1,131,524 (4.32 MB)

Trainable params: 1,131,524 (4.32 MB)

Non-trainable params: 0 (0.00 B)

Epoch 1/50

45/45  14s 273ms/step - accuracy: 0.3443 - loss: 1.0

Epoch 2/50

45/45  20s 262ms/step - accuracy: 0.4637 - loss: 1.0

Epoch 3/50

45/45  20s 248ms/step - accuracy: 0.5726 - loss: 1.0

Epoch 4/50

45/45  11s 255ms/step - accuracy: 0.6411 - loss: 0.9

Epoch 5/50

45/45  12s 257ms/step - accuracy: 0.7039 - loss: 0.8

Epoch 6/50

45/45  0s 219ms/step - accuracy: 0.7674 - loss: 0.6

Epoch 6: ReduceLROnPlateau reducing learning rate to 0.0005000000237487

45/45  20s 249ms/step - accuracy: 0.7730 - loss: 0.6

Epoch 7/50

45/45  10s 223ms/step - accuracy: 0.8226 - loss: 0.5

Epoch 8/50

45/45  12s 274ms/step - accuracy: 0.8610 - loss: 0.4

Epoch 9/50

45/45  0s 257ms/step - accuracy: 0.8573 - loss: 0.3

Epoch 9: ReduceLROnPlateau reducing learning rate to 0.0002500000118743

45/45  12s 273ms/step - accuracy: 0.8638 - loss: 0.3

Epoch 10/50

45/45  20s 260ms/step - accuracy: 0.9015 - loss: 0.2

Epoch 10: early stopping

Restoring model weights from the end of the best epoch: 3.

12/12  1s 68ms/step

CNN Test Accuracy: 0.5333

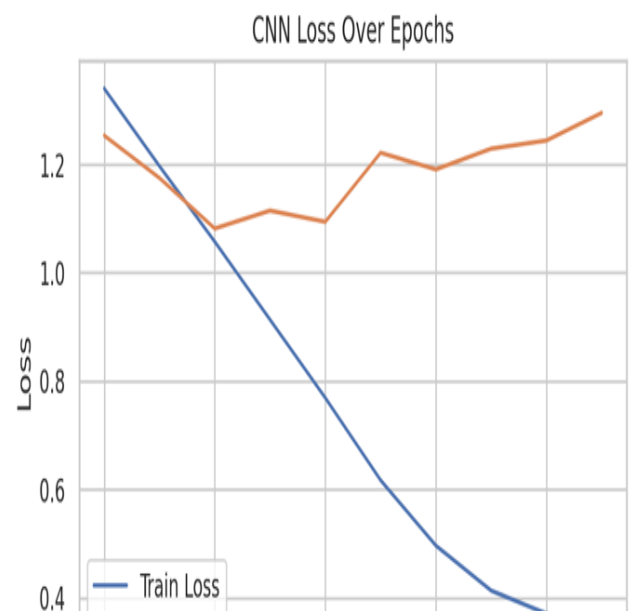
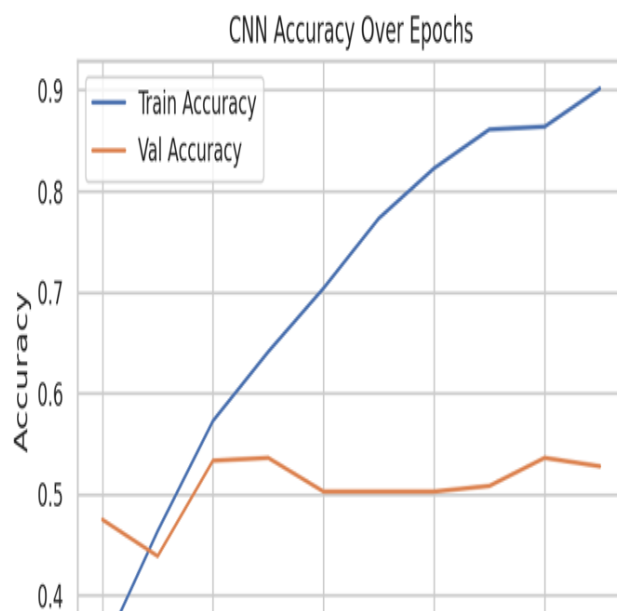
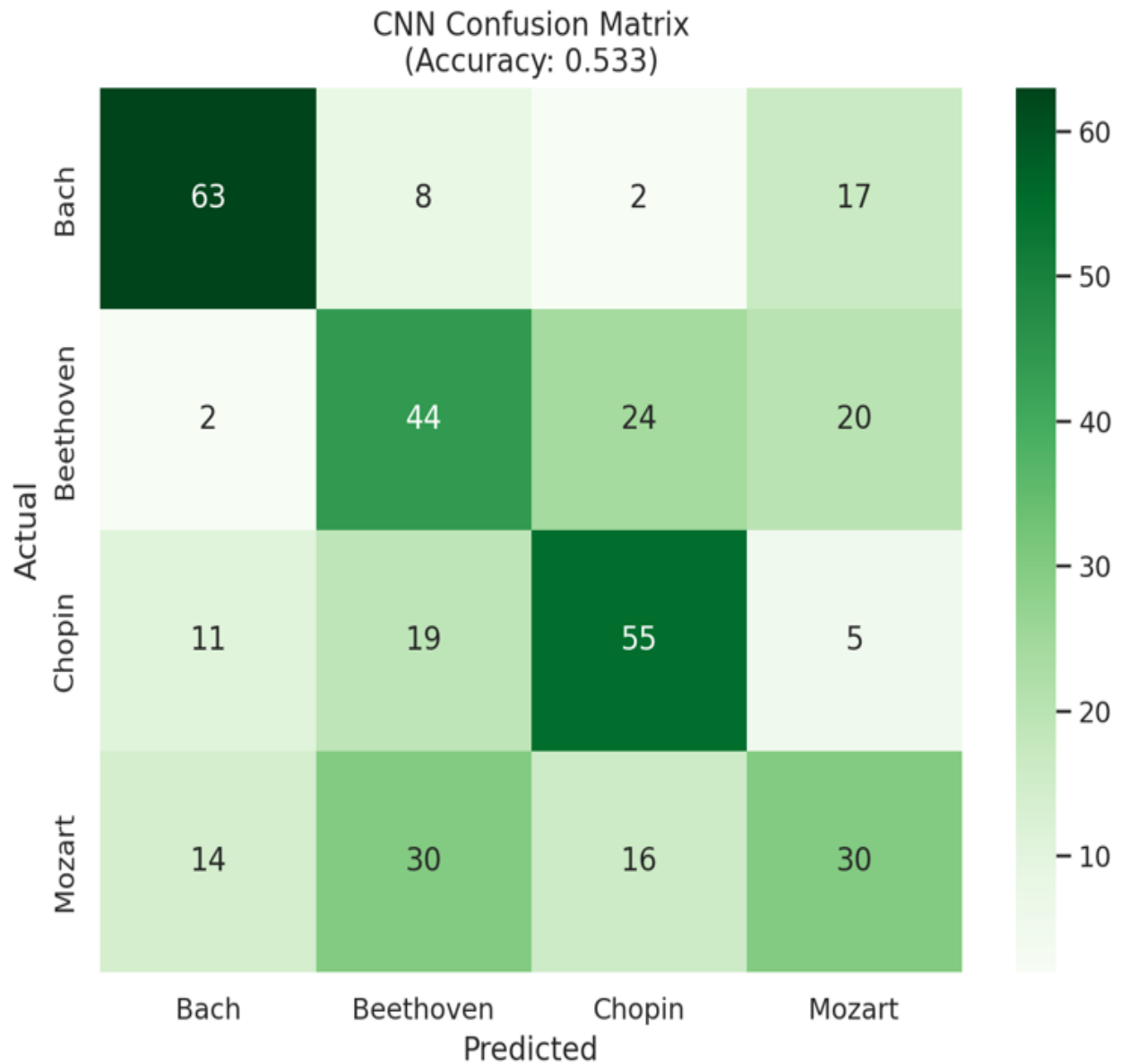
CNN Classification Report:

	precision	recall	f1-score	support
Bach	0.70	0.70	0.70	90
Beethoven	0.44	0.49	0.46	90
Chopin	0.57	0.61	0.59	90
Mozart	0.42	0.33	0.37	90
accuracy			0.53	360
macro avg	0.53	0.53	0.53	360
weighted avg	0.53	0.53	0.53	360

CNN Confusion Matrix:

[[63 8 2 17]

```
[ 2 44 24 20]
[11 19 55  5]
[14 30 16 30]]
```



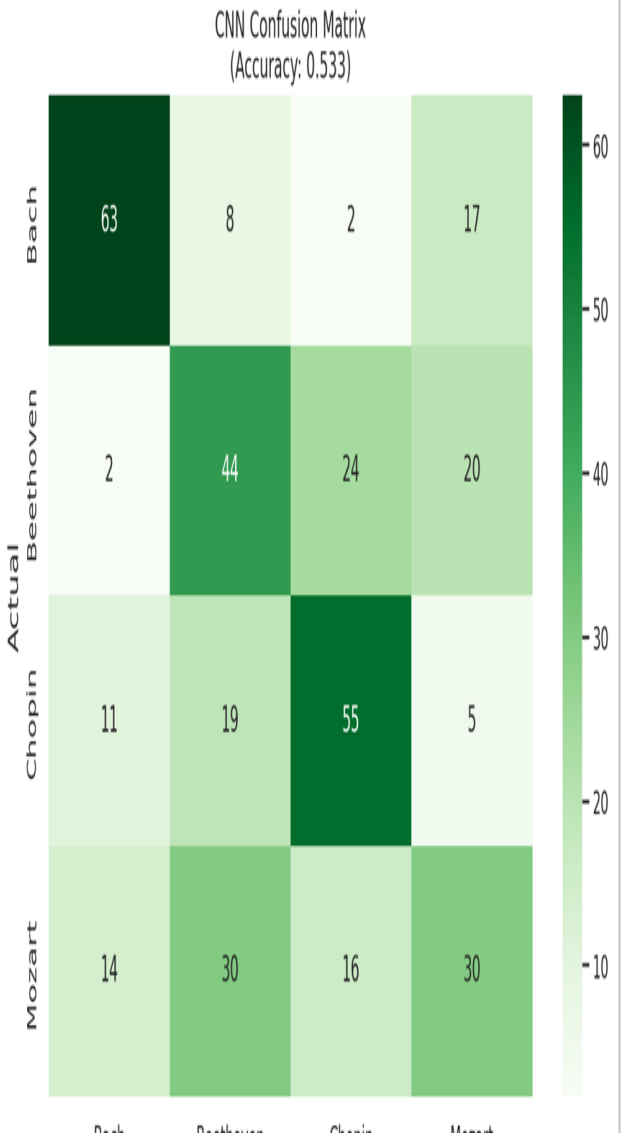
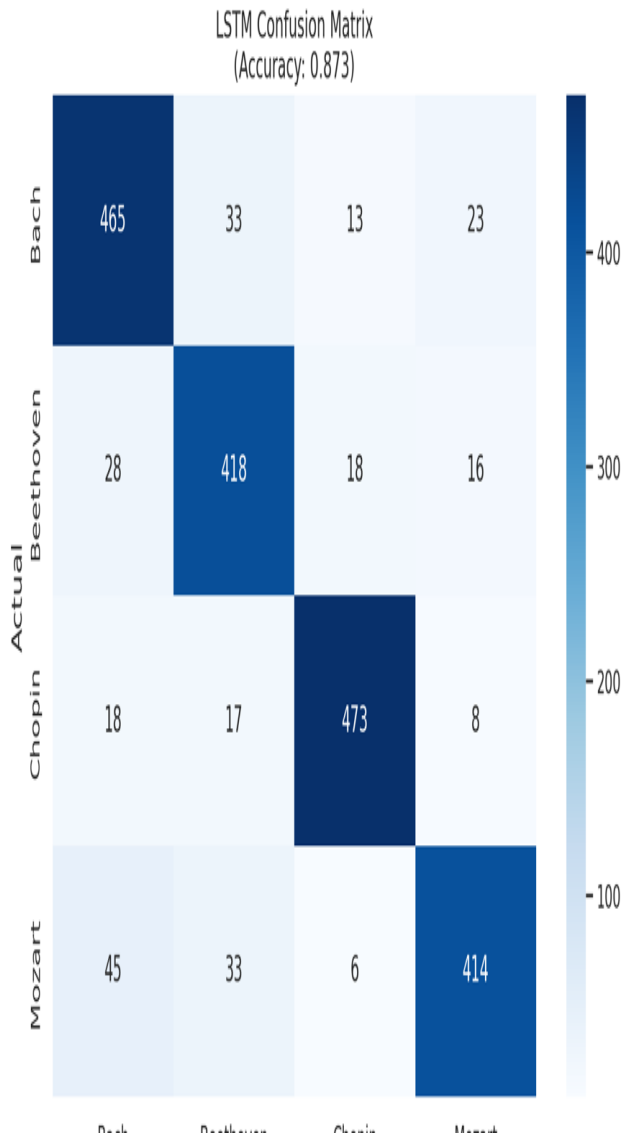


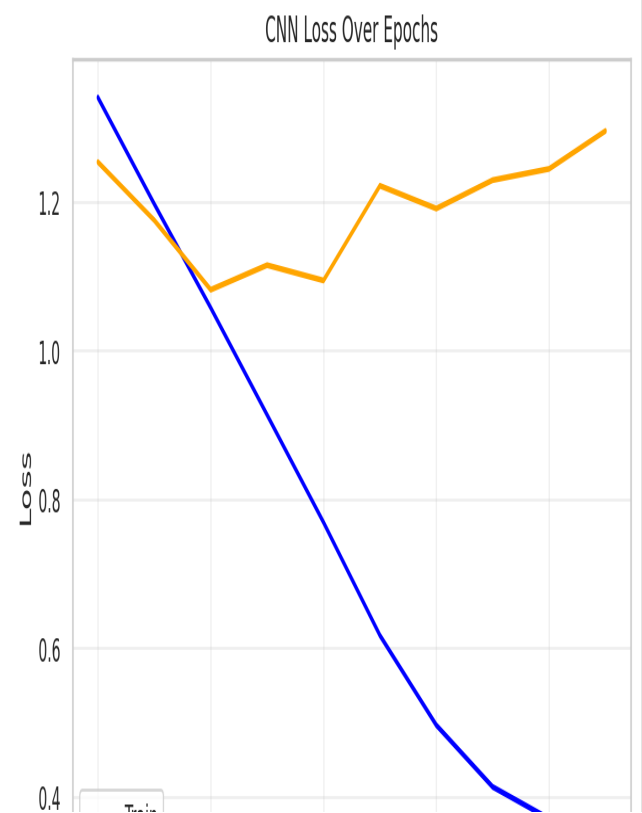
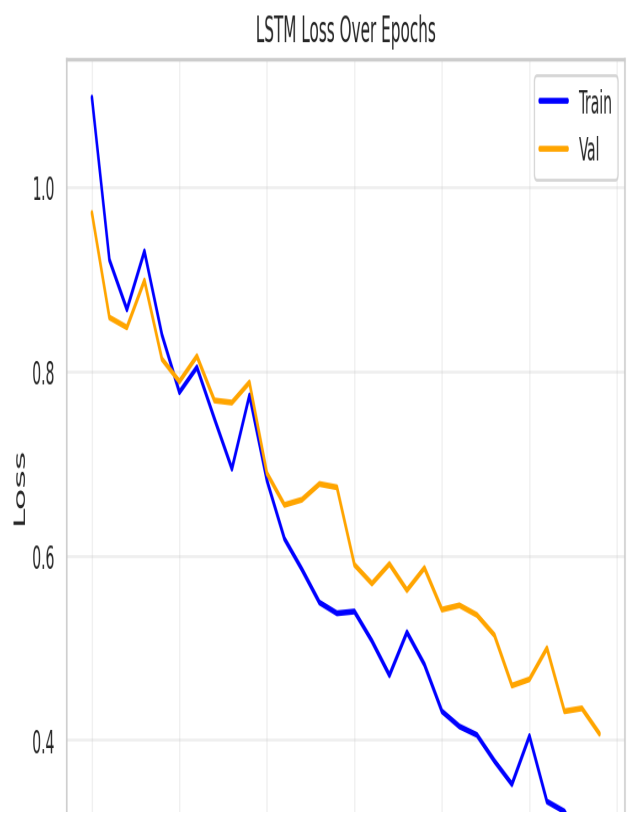
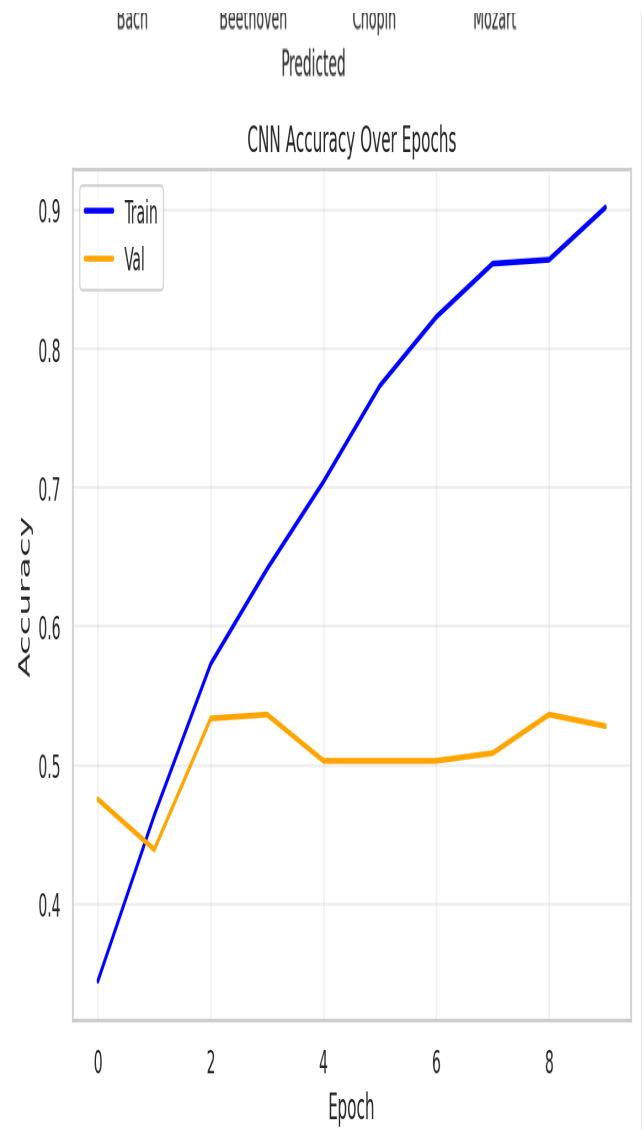
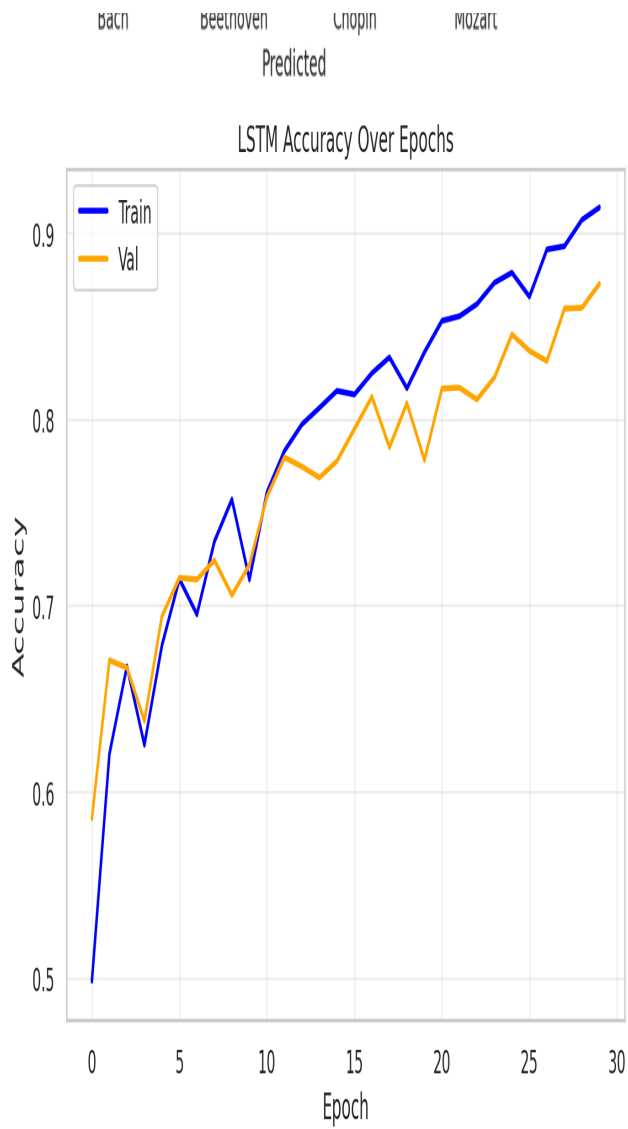
OVERALL ACCURACY COMPARISON

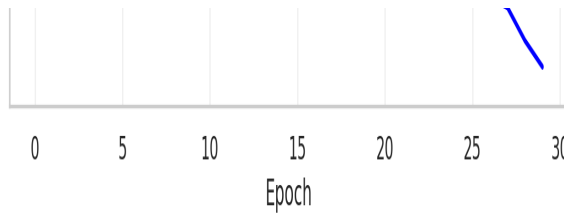
LSTM Test Accuracy: 0.8728 (87.28%)
CNN Test Accuracy: 0.5333 (53.33%)
Difference: -0.3394 (-33.94%)
LSTM outperforms CNN by 33.94%

PER-COMPOSER METRICS (Precision, Recall, F1-Score)

Composer	LSTM Precision	CNN Precision	LSTM Recall	CNN Recall	LSTM F1-Score
Bach	0.836331	0.700000	0.870787	0.700000	0.853333
Beethoven	0.834331	0.435644	0.870833	0.488889	0.852381
Chopin	0.927451	0.567010	0.916667	0.611111	0.922222
Mozart	0.898048	0.416667	0.831325	0.333333	0.863333







=====

SUMMARY INSIGHTS

=====

1. Overall Winner: LSTM by 33.94% accuracy
2. Mozart Classification:
 - LSTM: 83.1% recall
 - CNN: 33.3% recall
3. Model Characteristics:
 - LSTM: Learns sequential patterns in note/chord progressions
 - CNN: Learns spatial patterns in piano roll (harmony, texture)

=====

